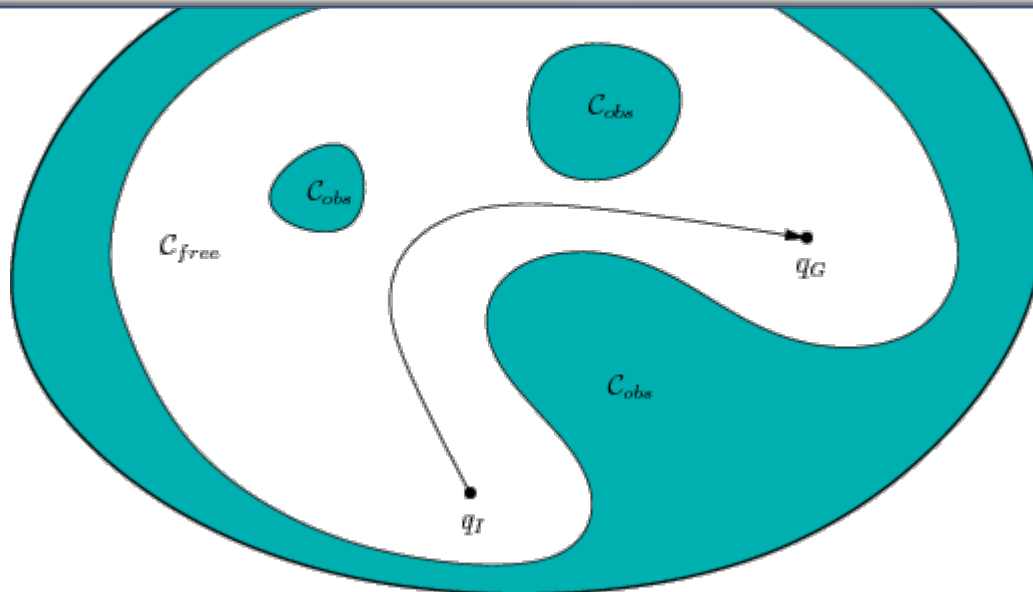


Riadenie mobilných robotov

*Globálna reprezentácia prostredia
+ globálna navigácia na topologickej mape*



/ musíme sa zaoberať
opisom miestom

Konfiguračné priestory

Konfiguračný priestor

- **definícia:** množina všetkých dosiahnuteľných polôh robota v priestore

→ moduluje počet stupňov voľnosti

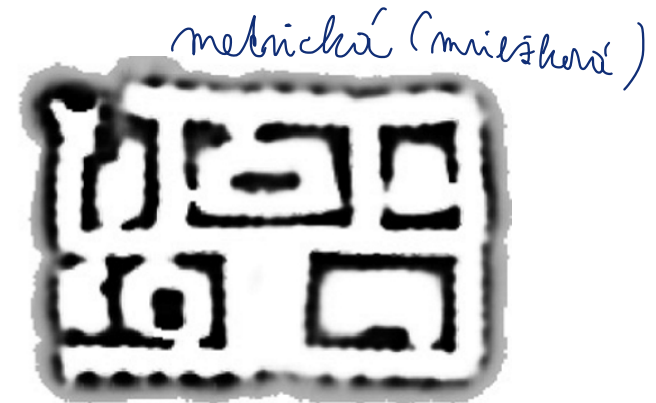
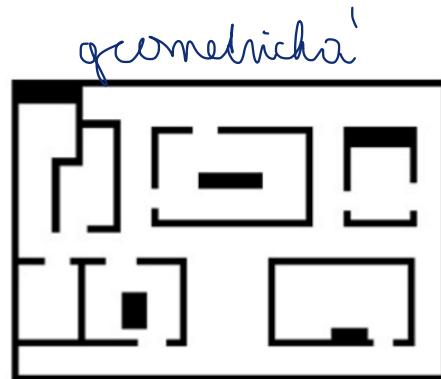
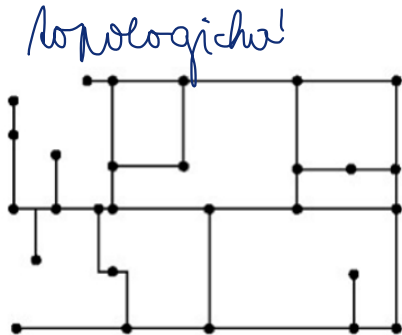
- **typy konfiguračných priestorov:**
 - geometrická mapa
 - metrická mapa (bunková, mriežková)
 - topologická mapa
 - hybridná mapa

Poloha v konfiguračnom priestore

- **definícia:** bod alebo množina bodov v konfiguračnom priestore
- dobre navrhnutý konfiguračný priestor redukuje počet stupňov voľnosti, s ktorými počíta globálna navigácia

Mapa priestoru

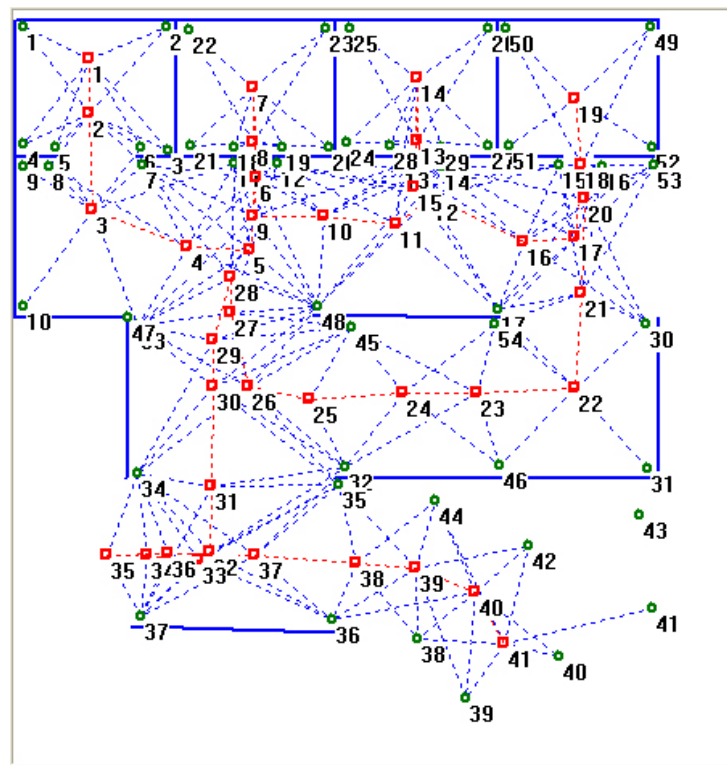
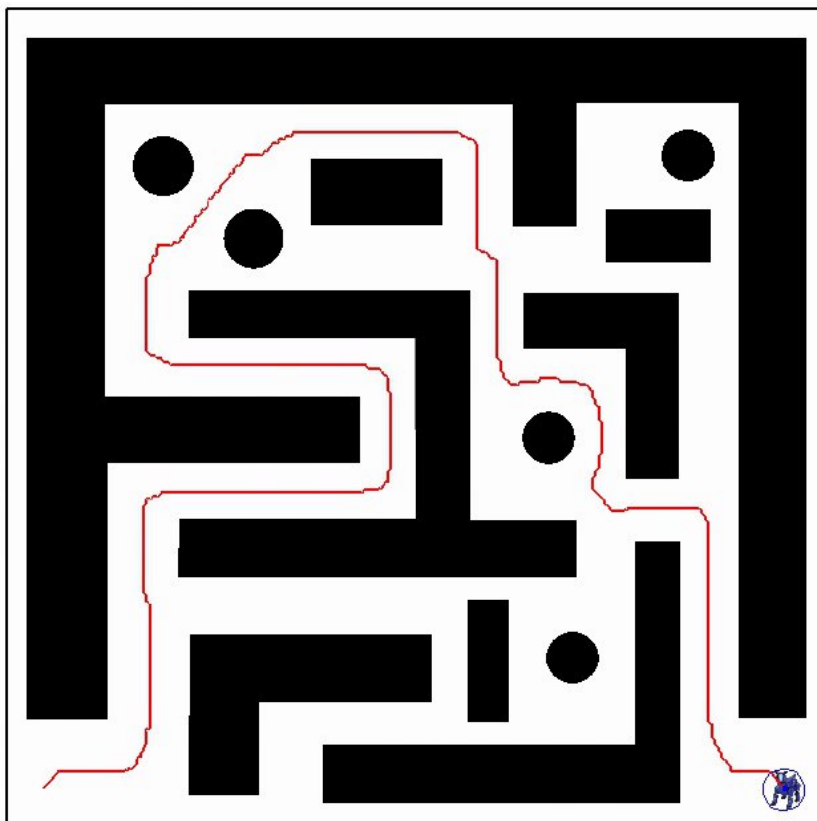
- **definícia:** zmenšený obraz povrchu Zeme alebo jeho určitej časti (vo všeobecnosti)
- mapa v robotike reprezentuje povrch (priestor), v ktorom sa robot pohybuje



Geometrická mapa

- **definícia:** reprezentácia prostredia, ktorá na reprezentáciu používa základné geometrické tvary
→ je možnosť ale často sa tam nachádza dráha !
- geometrickými tvarmi sú znázornené prekážky alebo nepriechodné časti prostredia
- priestor medzi geometrickými tvarmi je voľný, t. j. priechodný
- čiarové segmenty, polygóny, trojuholníkové siete

Geometrická mapa



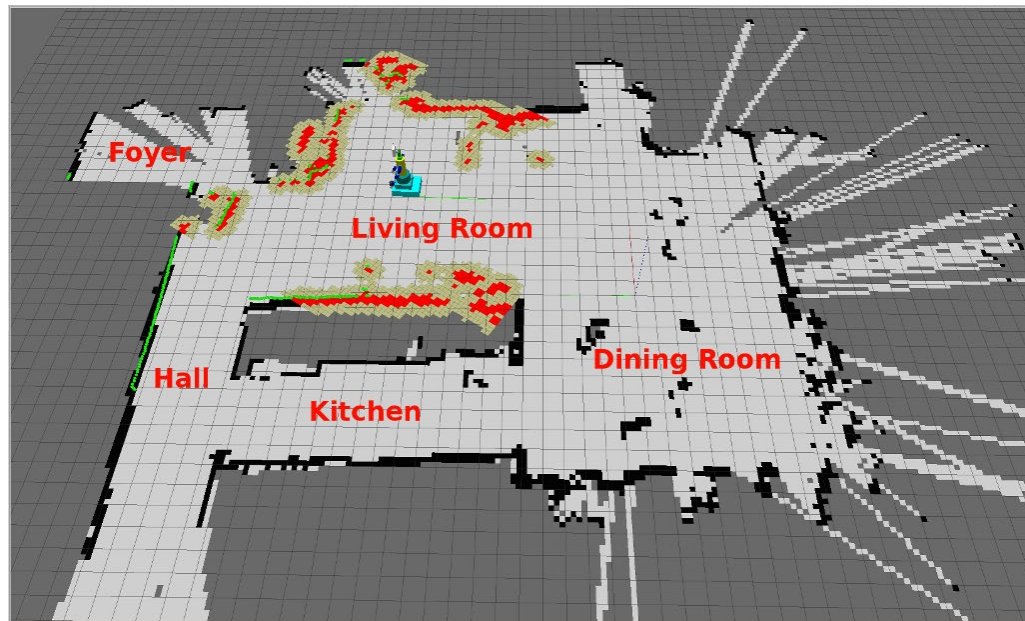
- stena
- entita prostredia
- poloha robota počas prieskumu
- spojnica charakterizujúca viditeľnosť entity v danej polohe robota
- spojnica polôh charakterizujúca postupnosť polôh

Metrická mapa

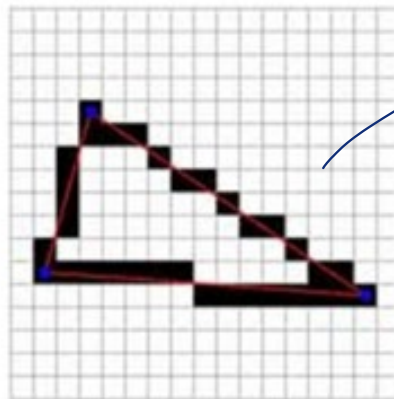
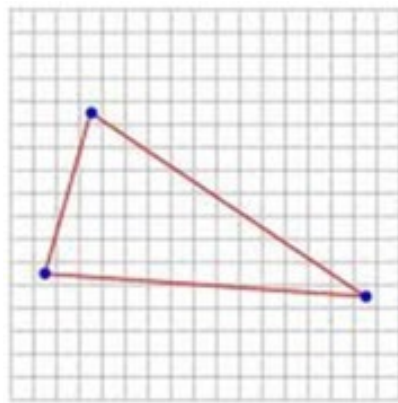
musíme sa
makať ale ľahko
sa v nej pohybuje

- **definícia:** reprezentácia prostredia, ktorá na reprezentáciu používa mriežku tvorenú z buniek (bunková alebo mriežková)
- využíva metriku, t. j. súbor pravidiel na určenie vzdialenosti medzi dvoma bodmi priestoru
- bunky sú reprezentované pravdepodobnosťou vyjadrujúcou mieru istoty, že bunka je nepriechodná, resp. priechodná
- schopnosť reprezentovať aj nespoznané prostredie
- často sa tvoria z geometrických alebo redukovujú na topologické

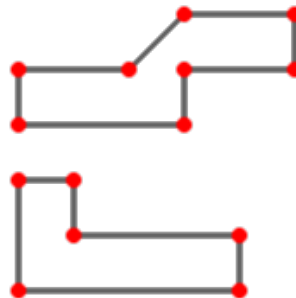
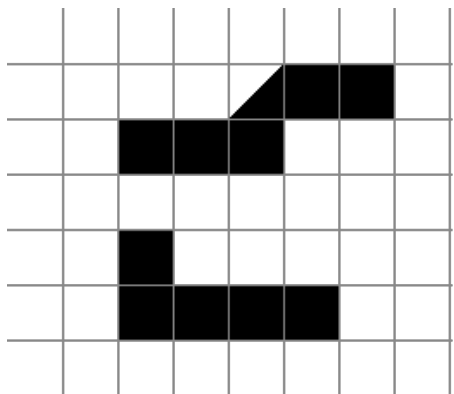
Metrická mapa



Metrická a geometrická mapa



deformácia
maložky !



Tvorba metrickej mapy (SLAM)



iCLEBO
INTELLIGENT CLEANING ROBOT
Arte



iCLEBO Arte
Robot Mapping

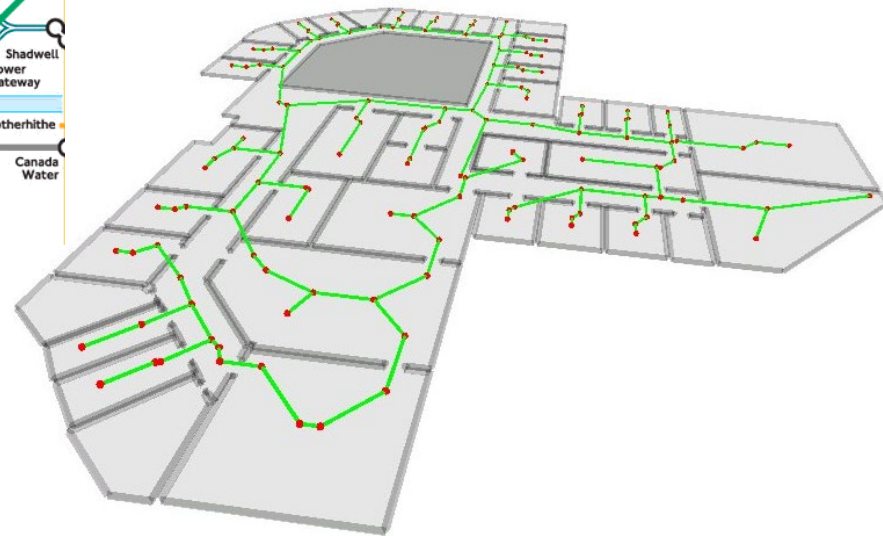
Topologická mapa

- **definícia:** reprezentácia prostredia, ktorá na reprezentáciu používa grafové štruktúry

→ jednoduchá štruktúra → jednoduché plánovanie ale neposkytne
relácie medzi (rozmery)

- vrcholy grafu = jednoznačne určené miesta v prostredí
- hrany grafu = prechody medzi vrcholmi (obvykle neohodnotené)
- vhodné na reprezentáciu veľkého priestoru (obvykle kompozícia z metrických máp)

Topologická mapa



Topologická mapa

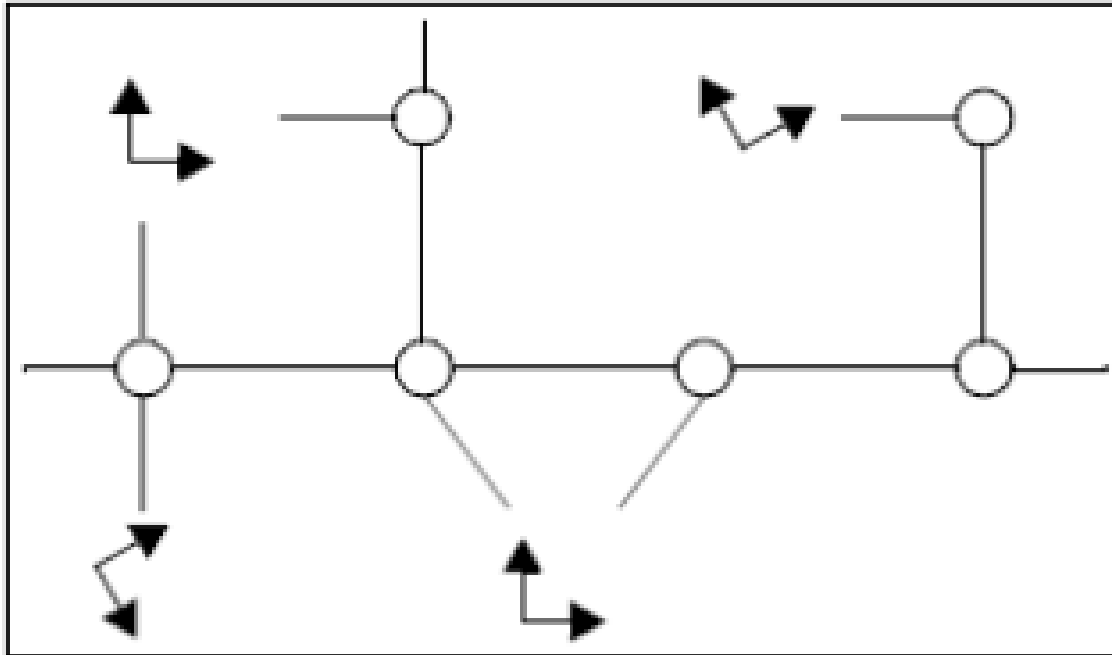


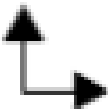
Hybridná mapa

- kombinuje rôzne mapy z dôvodu využitia výhod jednotlivých máp
- napr. metrické a topologické mapy (využíva zachovanie metrických vzťahov a ľahkého plánovania z topologických máp)

↳ lebo obsiahneme veľký priestor ale normálne
lokálnu metrickú mierku

Hybridná mapa



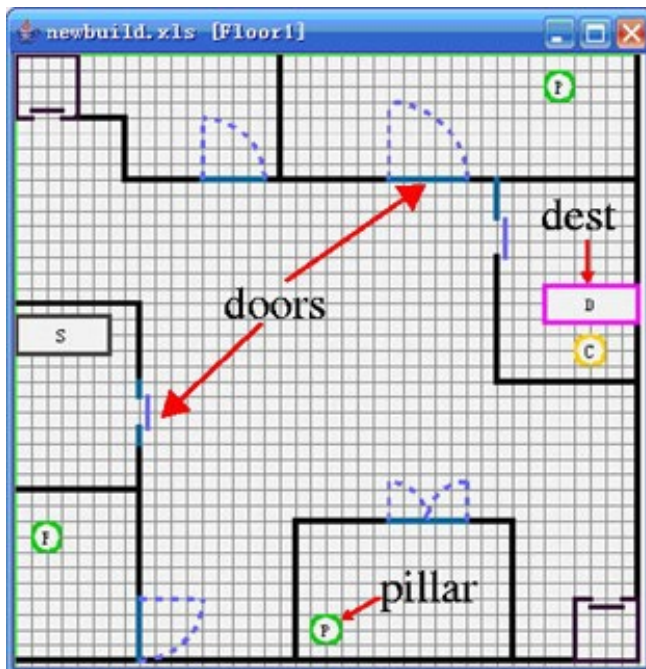
 Lokálne metrické miesto

 Topologický uzol

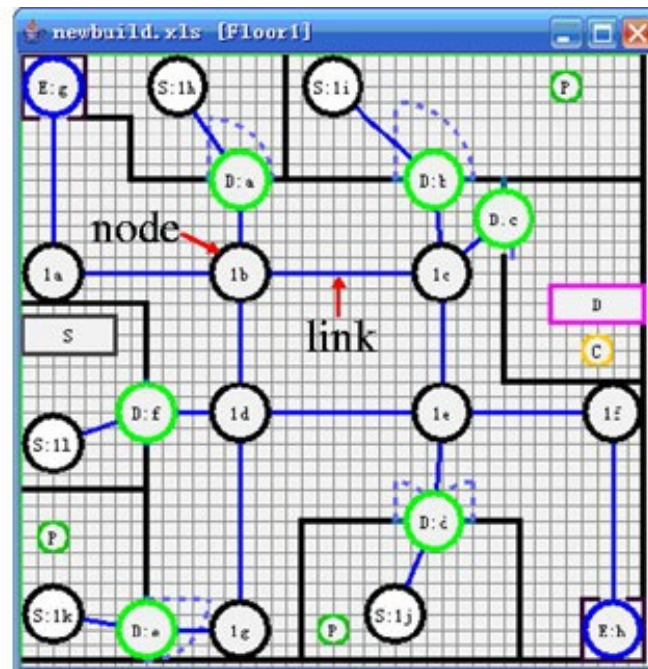
 Topologická hrana

 Hrana z topologickej do metri cej mapy

Hybridná mapa

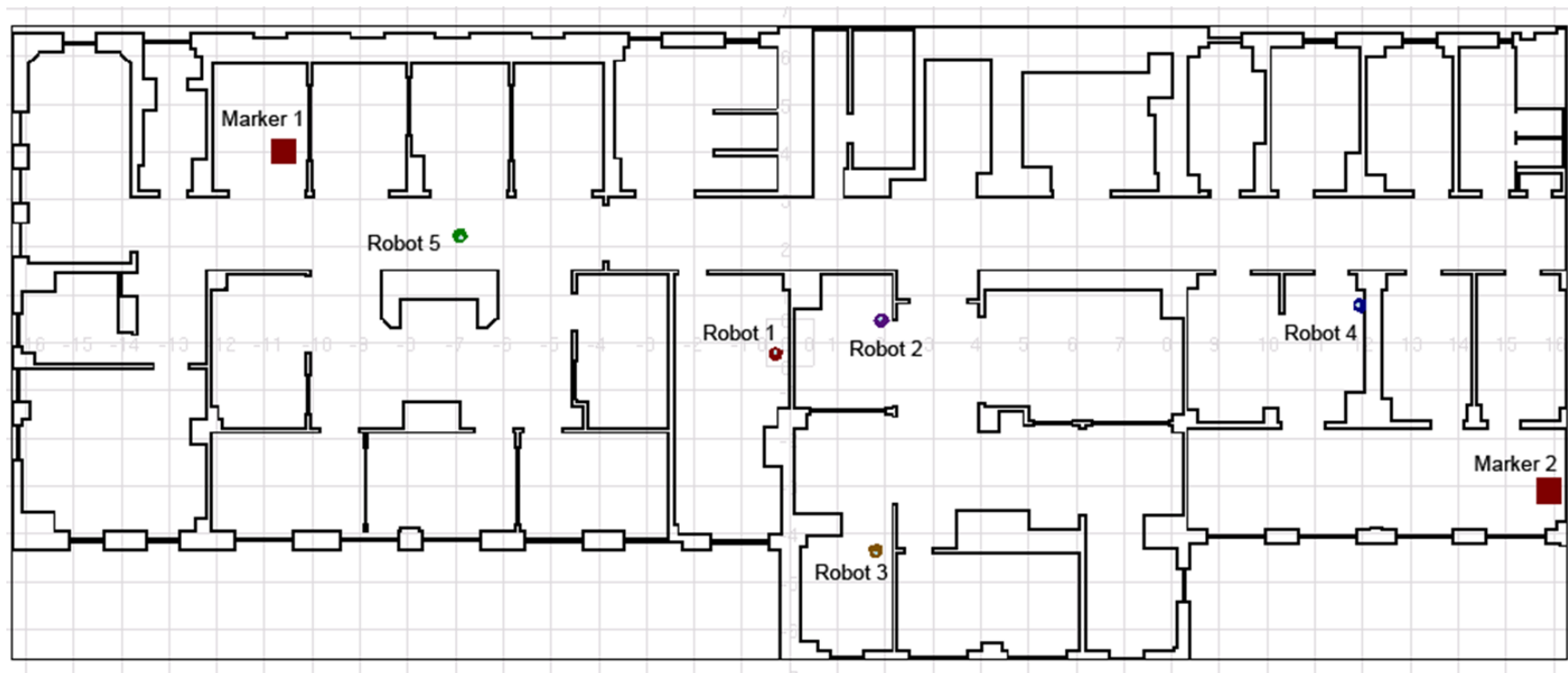


(a)



(b)

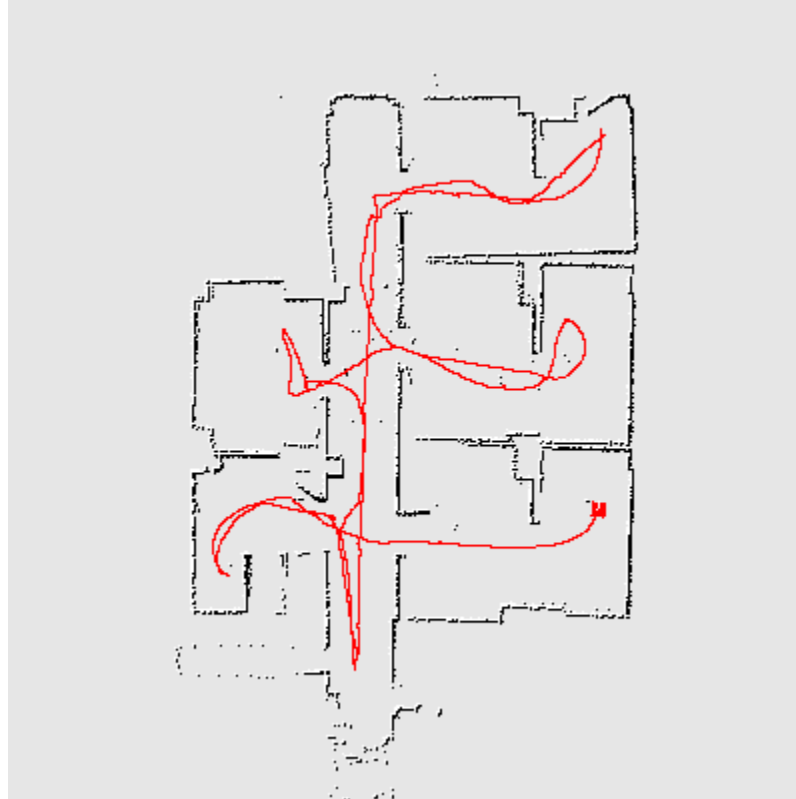
Hybridná mapa



Mapy z hľadiska rozsahu

- lokálne — *ne reaktívne navigácie*
 - obmedzené meracím rozsahom snímačov robota
- globálne
 - pokrývajú celý pracovný priestor robota
 - obvykle vznikajú spájaním lokálnych máp

Mapovanie priestoru



Z geometrických na metrické a topologické

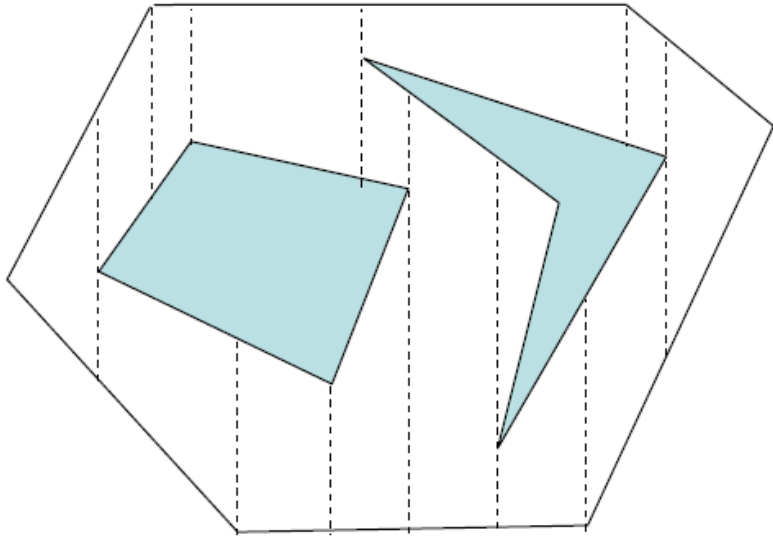
29. 4. 2025

Tvorba máp z geometrických na metrické

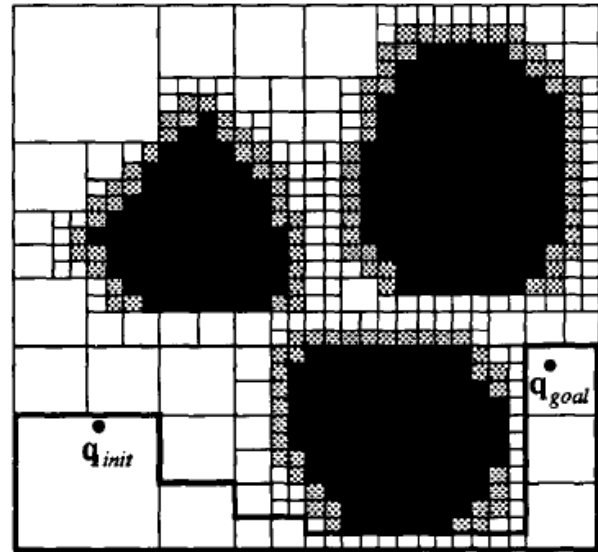
- bunková dekompozícia; **definícia**: rozdelenie geometricky definovaného priestoru na metrické úseky (bunky)
- **exaktná bunková dekompozícia** = ak sú hranice buniek súčasťou štruktúry prostredia (napr. stien prekážok), tzv. **bezstratová dekompozícia**
- **približná bunková dekompozícia** = ak sú bunky len priblížením aktuálnej štruktúry prostredia

Exaktná a približná BD

exaktná dekompozícia



struktúra

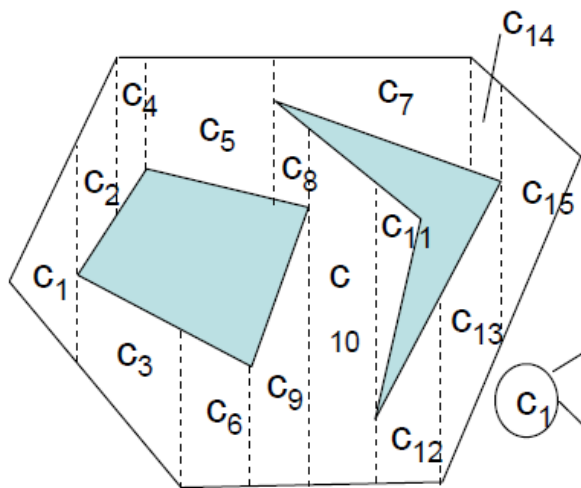


Exaktná bunková dekompozícia (EBD)

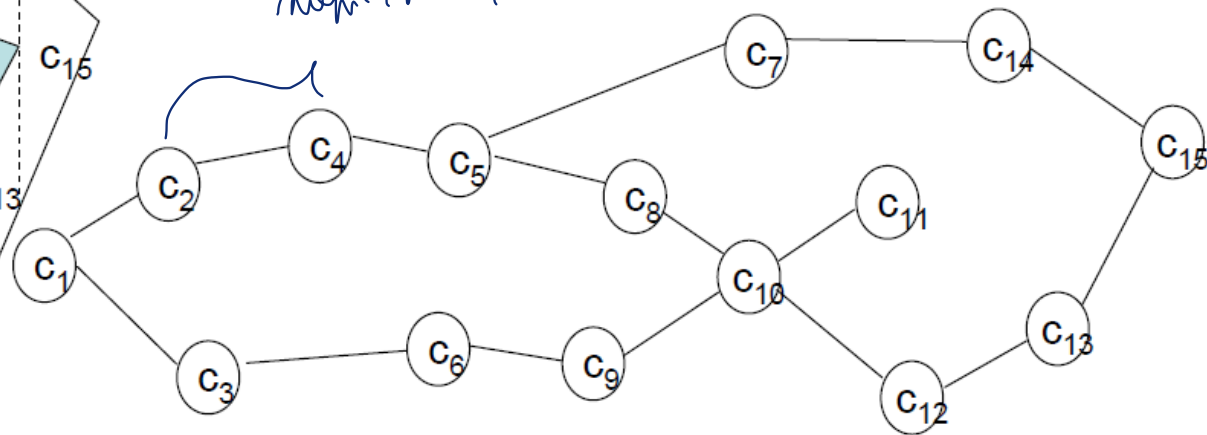
- hranice buniek sú tvorené geometrickými vlastnosťami prostredia
- bunky sú ohodnotené na priechodné a nepriechodné
- rôzne metódy
- na nájdenie prechodu prostredím sa vytvára tzv. **graf susednosti** (vrcholy sú tvorené priechodnými bunkami a hrany prechodmi medzi týmito bunkami)

"vieme prejsť z jednej
bunky do druhej; ako
sa 'vedúť'
sela"

Graf susednosti



napr. nie je funkcia 2 susedi → funkcion 4



↳ je to topologická reprezentácia miestom

Algoritmus nájdania cieľa pomocou grafu susednosti

1. rozdel' priestor na jednoduché spojené úseky, tzv. bunky
2. rozhodni, ktoré bunky sú voľné a jednotlivé voľné bunky spoj do grafu susednosti
3. nájdí bunky, v ktorých je štartovacia a cieľová pozícia a nájdí cestu v grafe tak, aby si spojil tieto bunky
4. z vybranej sekvencie buniek v grafe urči dráhu predpisom stredných bodov buniek alebo sledovaním obrysov v jednotlivých bunkách

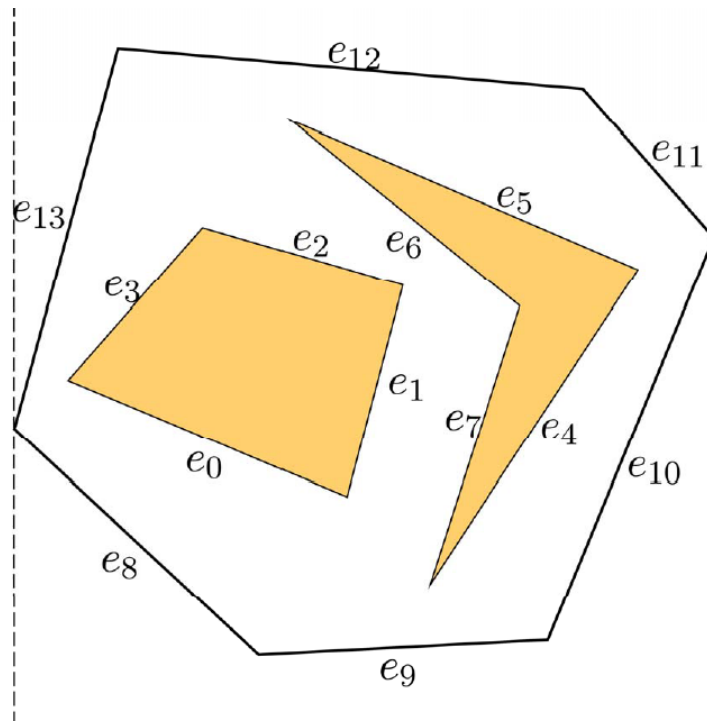
Lichobežníková EBD

dekompozícia

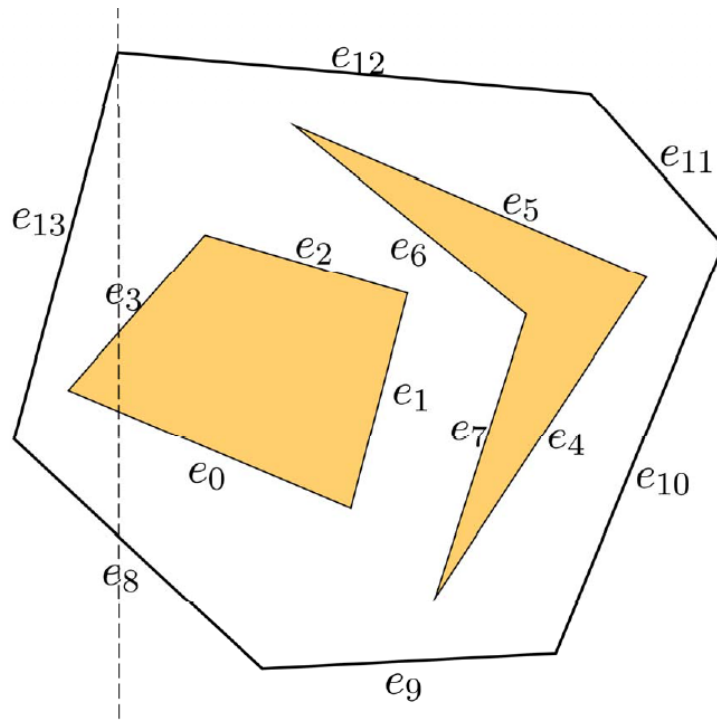
⇒ budú vzniknúť lichobežníky

- pre každý vrchol v_i z geometrickej reprezentácie prostredia sa vytvorí horné a dolné vertikálne predĺženie
- **definícia:** predĺženie je pomyselná spojnica medzi vrcholom prekážky a inou prekážkou alebo hranicou priestoru, orientovaná v smere osí globálneho súradnicového systému

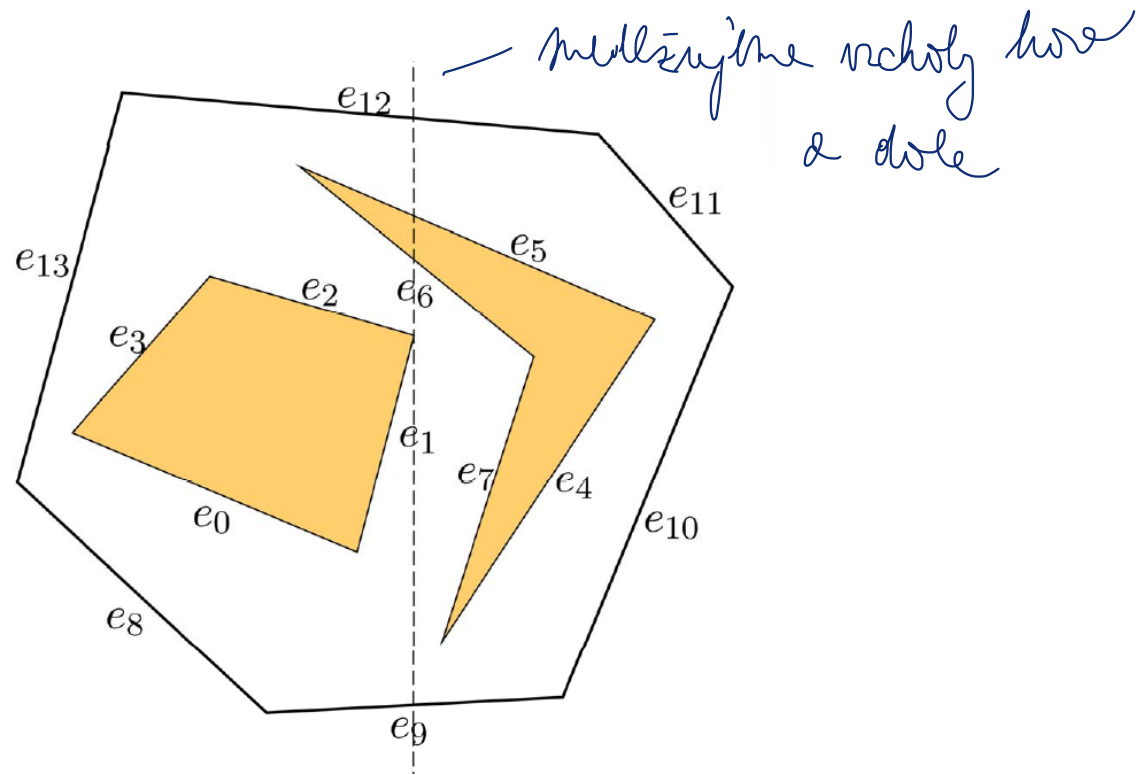
Lichobežníková EBD



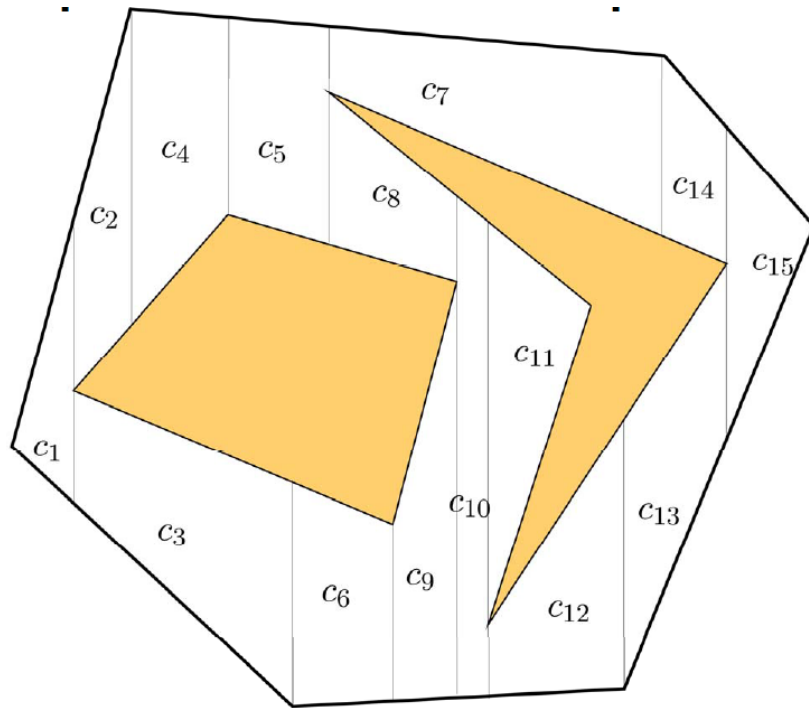
Lichobežníková EBD



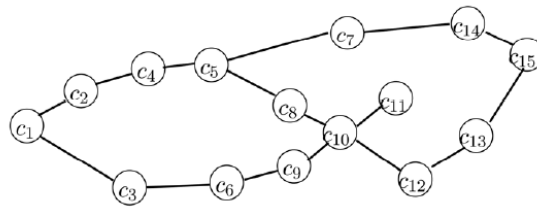
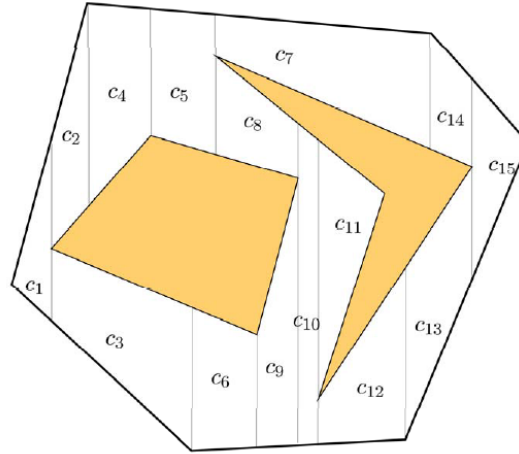
Lichobežníková EBD



Lichobežníková EBD

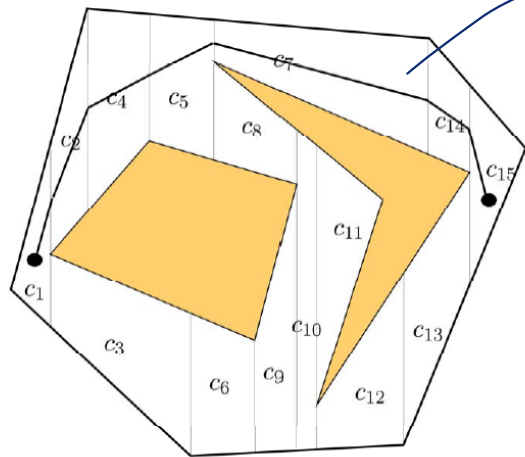


Lichobežníková EBD

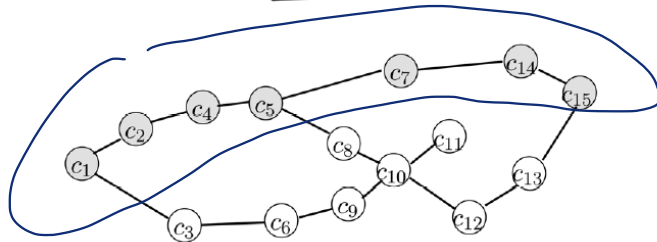


- graf susednosti
na ktorom vieme
použiť metodu

Lichobežníková EBD – najdenie dráhy

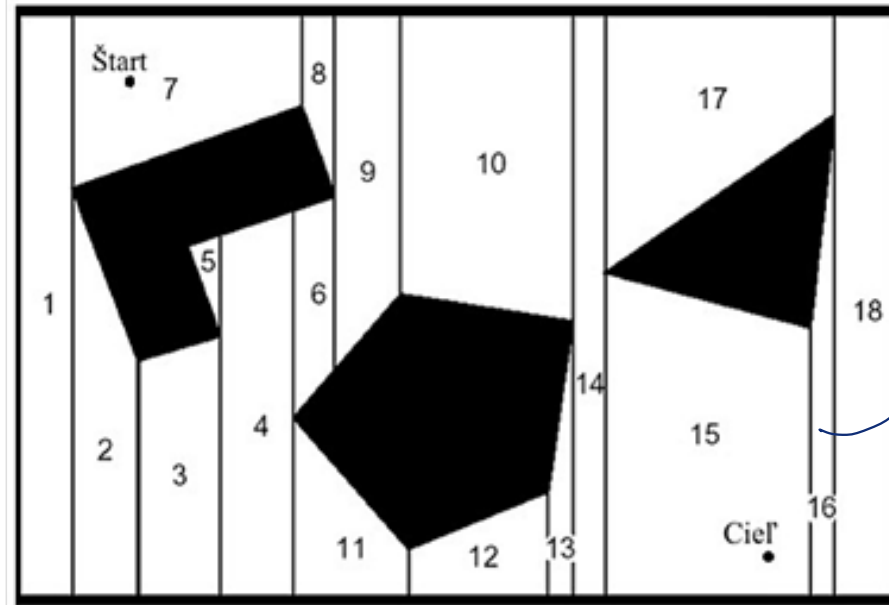


prechádzame dĺžkou
hraniek $\Rightarrow$ najkratší
prechod

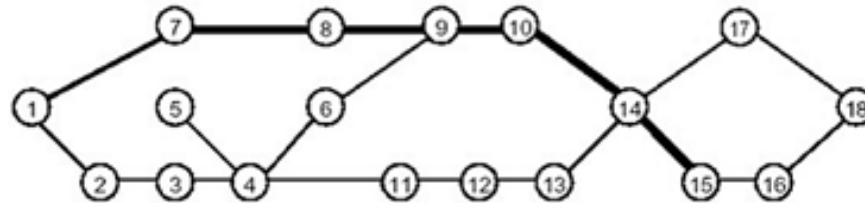


prechod

Lichobežníková EBD

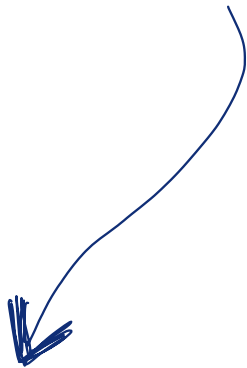


*medziaranie
len dole*



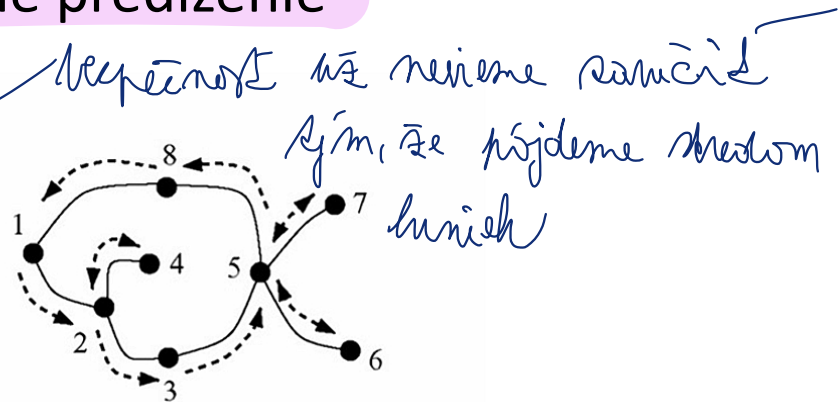
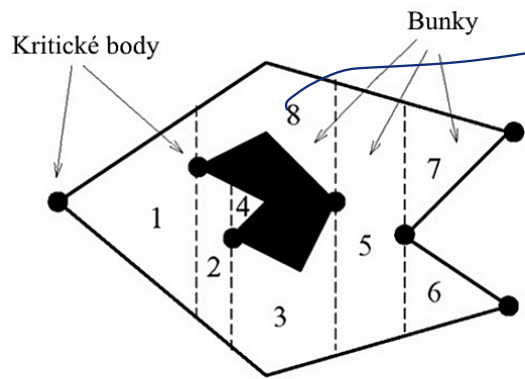
Lichobežníková EBD – vlastnosti

- lichobežník je konvexná množina → vo vnútri lichobežníkov možno voliť ľubovoľnú možnú dráhu → bezkolíznosť prechodu robota cez bunku
- príliš veľa buniek, ktoré možno spojiť do väčších celkov
- iné metódy EBD



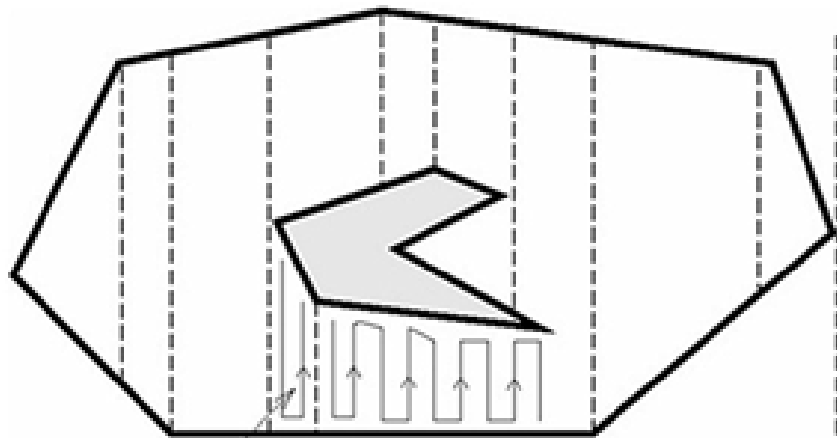
Zrkadlová (boustrophedon) EBD

- predĺženia len z kritických bodov
- **definícia:** kritický bod je taký vrchol, z ktorého je možné vytvoriť horné aj dolné vertikálne predĺženie

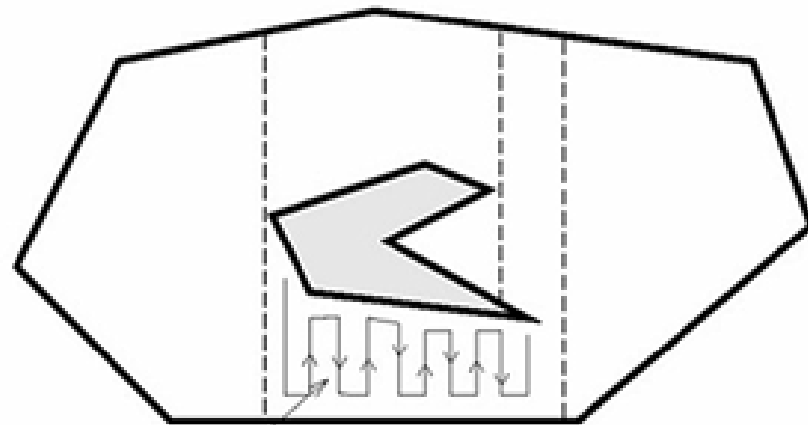


Úplné pokrytie 1-2-4-2-3-5-6-5-7-5-8-1

Lichobežníková a zrkadlová EBD



Dráha pokrytia v bunke.



Dráha pokrytia v bunke.

híme polýt celu' dráhu hunky

Morseova EBD

→ rezy môžu byť
rôznych funkcií

- rozdeľuje štruktúru prostredia pomocou funkcie definovanej na jednorozmernej rovine nazývanej rez
- rezy sú tvorené na základe kritických bodov
- rôzny charakter rezov
- Morseova EBD zaručuje, že medzi rezmi zostáva topológia rovnaká (t. j. nemení sa počet bodov prieniku obrysových kriviek a priamky tvoriacej smer rezu)

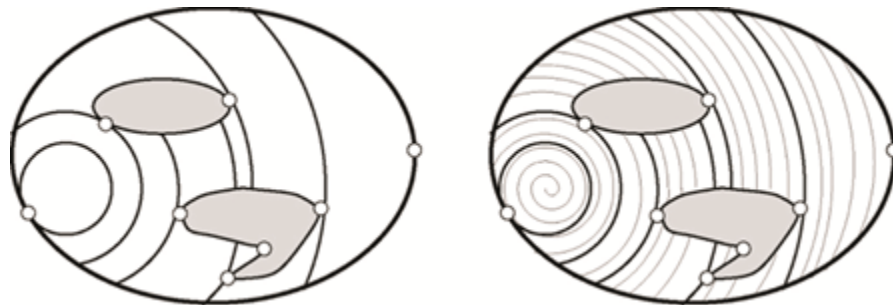
Pokrytie priestoru pomocou Morseovej EBD

- pohyb pozdĺž rezu (ukončený ak narazí na prekážku) a pohyb medzi hranicami prekážok
- dĺžka pohybu robota pri prekážke je definovaná šírkou robota, t. j. funkcia rezu je zväčšená o šírku robota

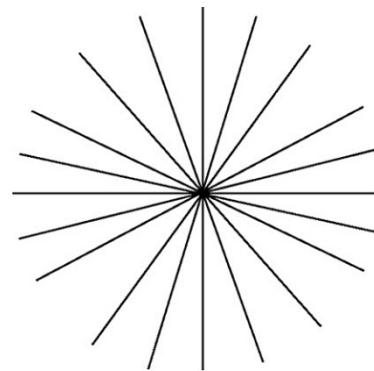
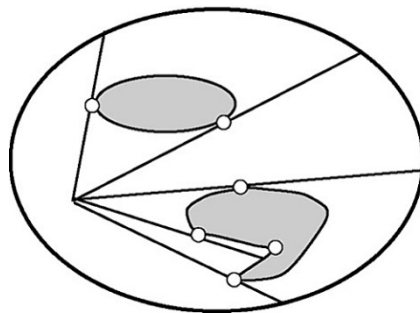
Morseova EBD

very r style
kružnice

$$h(x, y) = \sqrt{x^2 + y^2}$$

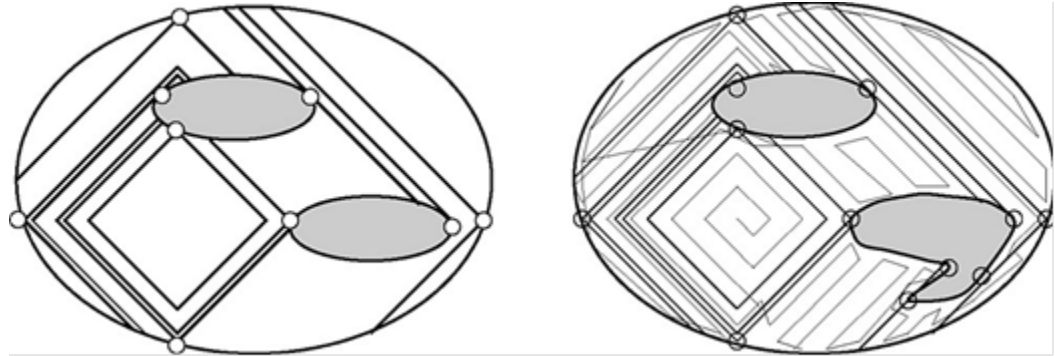


$$h(x, y) = \tan\left(\frac{y}{x}\right)$$



Morseova EBD

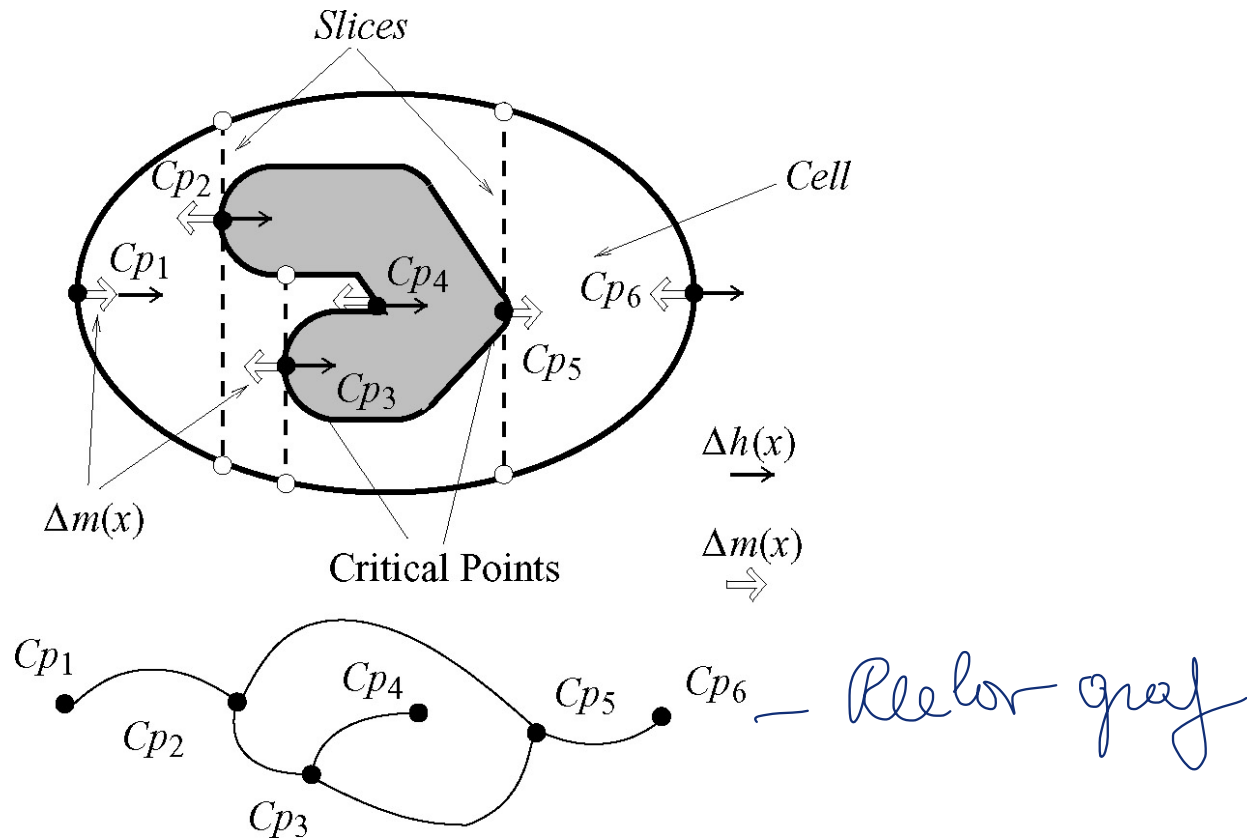
$$h(x, y) = |x| + |y|$$



EBD v neznámom prostredí

- dekompozícia pomocou pokrytia snímačmi, napr. boustrophedon (druh obojsmerného písma – každý druhý riadok je obrátený alebo písaný opačným smerom)
- duálny graf, tzv. **Reebov graf** (vrcholmi grafu sú kritické body a hrany tvoria spojenia medzi týmito kritickými bodmi)

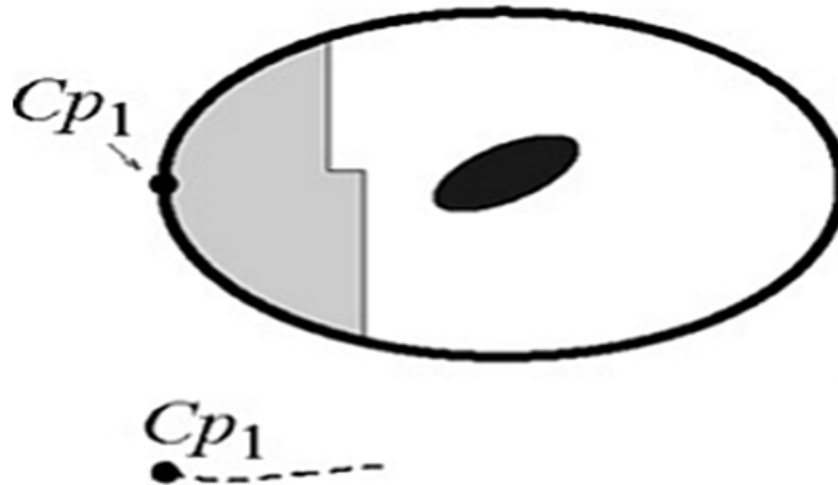
Boustrophedon dekompozícia



Reebov graf

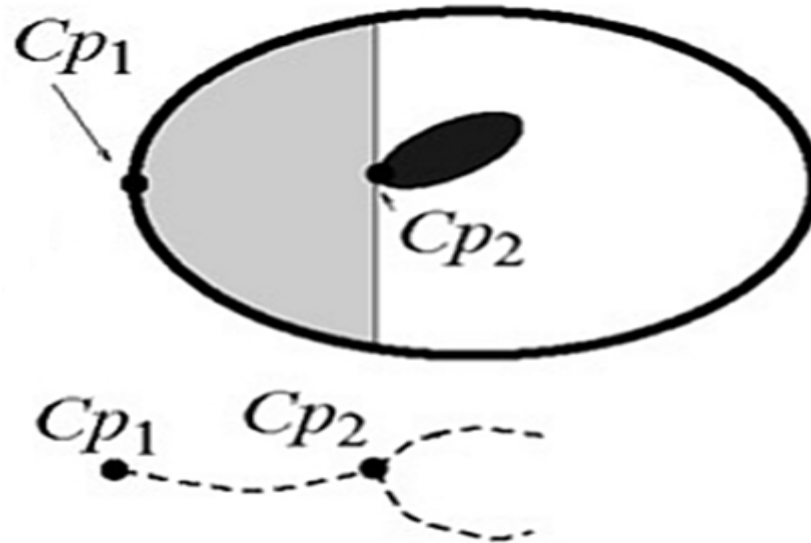
1. robot začne pokrývať priestor v kritickom bode Cp_1

byťm nemasazi na
dielši kritický bod



Reebov graf

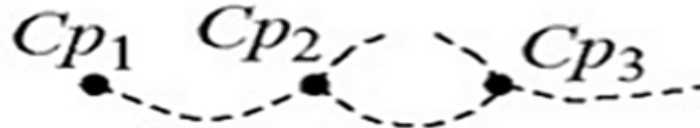
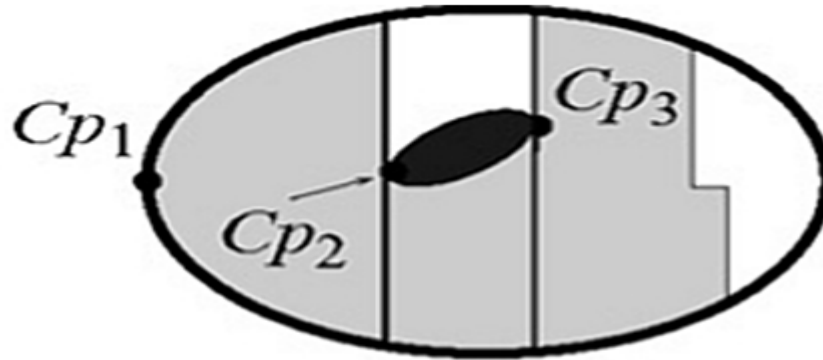
2. ak robot pokryje celú bunku medzi dvoma kritickými bodmi Cp_1 a Cp_2 , vytvorí odpovedajúce hrany



Reebov graf

ale vždy rovnake!

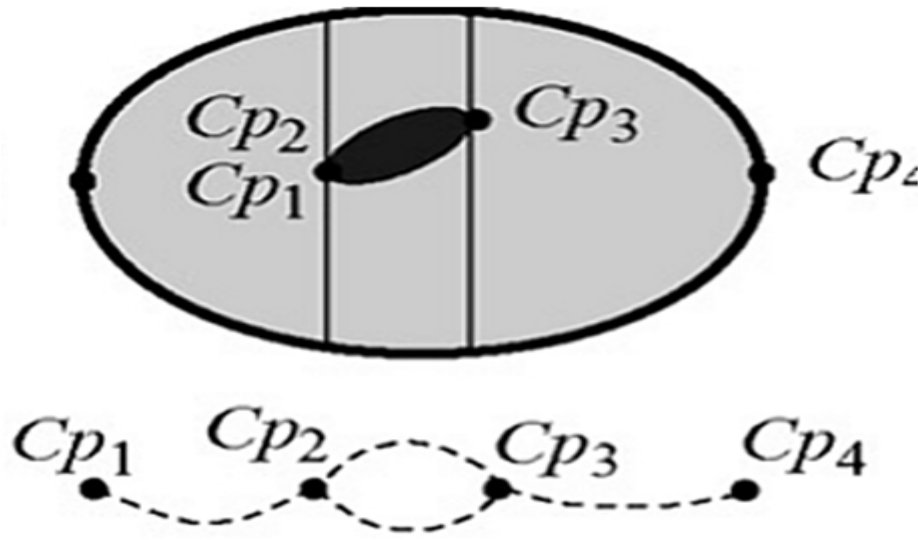
3. neznáme prostredie = robot vyberá ľubovoľnú stranu;
dosiahne kritický bod Cp_3 – bunka medzi Cp_2 a Cp_3 je pokrytá
– ďalej pokrýva bunku vpravo od Cp_3



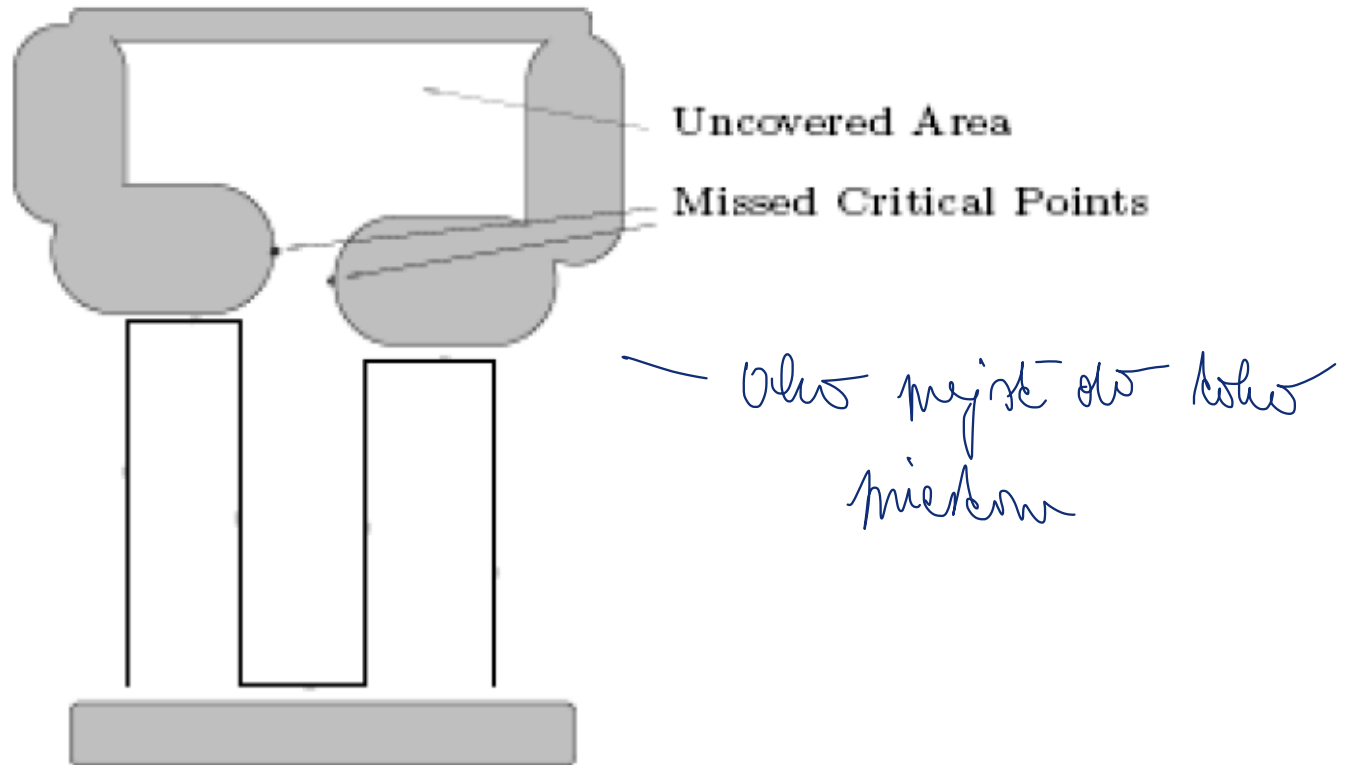
Reebov graf

→ zabezpečení, že najdeme všetky kritické body a ^{nikdy} nikdy ich ^{niekoľ}

4. narazí na Cp_4 , vracia sa na Cp_3 a pokryje bunku nahor od Cp_3 , potom sa vráti až na Cp_2 a overí, či boli pokryté všetky kritické body v prostredí



Problém



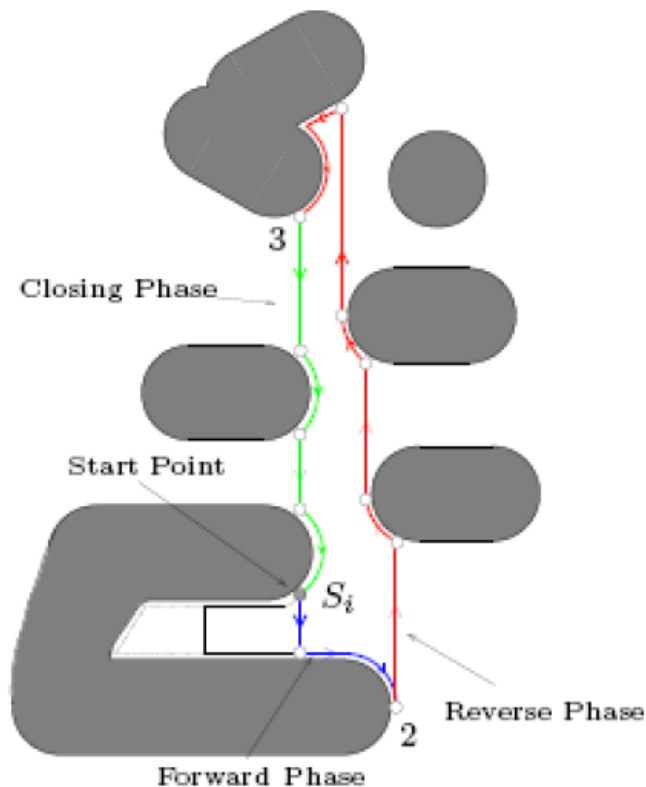
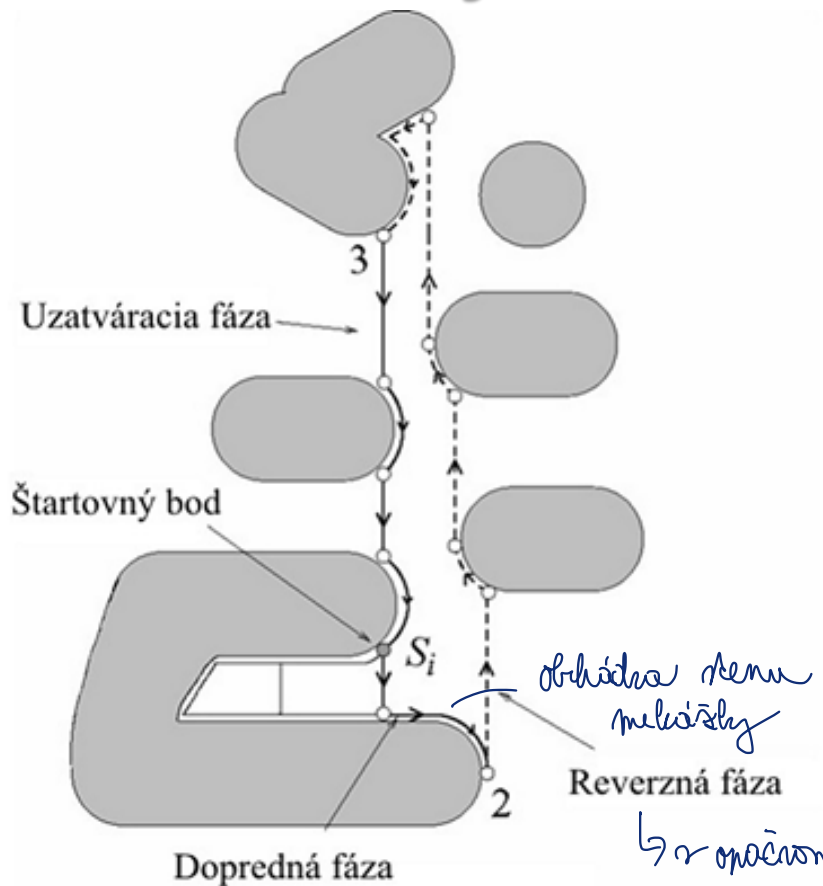
Reebov graf

- detekcia kritických bodov a zabezpečenie pokrytia všetkých kritických bodov!
- kritické body možno detegovať ako extrémny v prostredí, napr. normály prekážok na smer pohybu robota
- pokrytie všetkých kritických bodov → algoritmus (3 fázy)

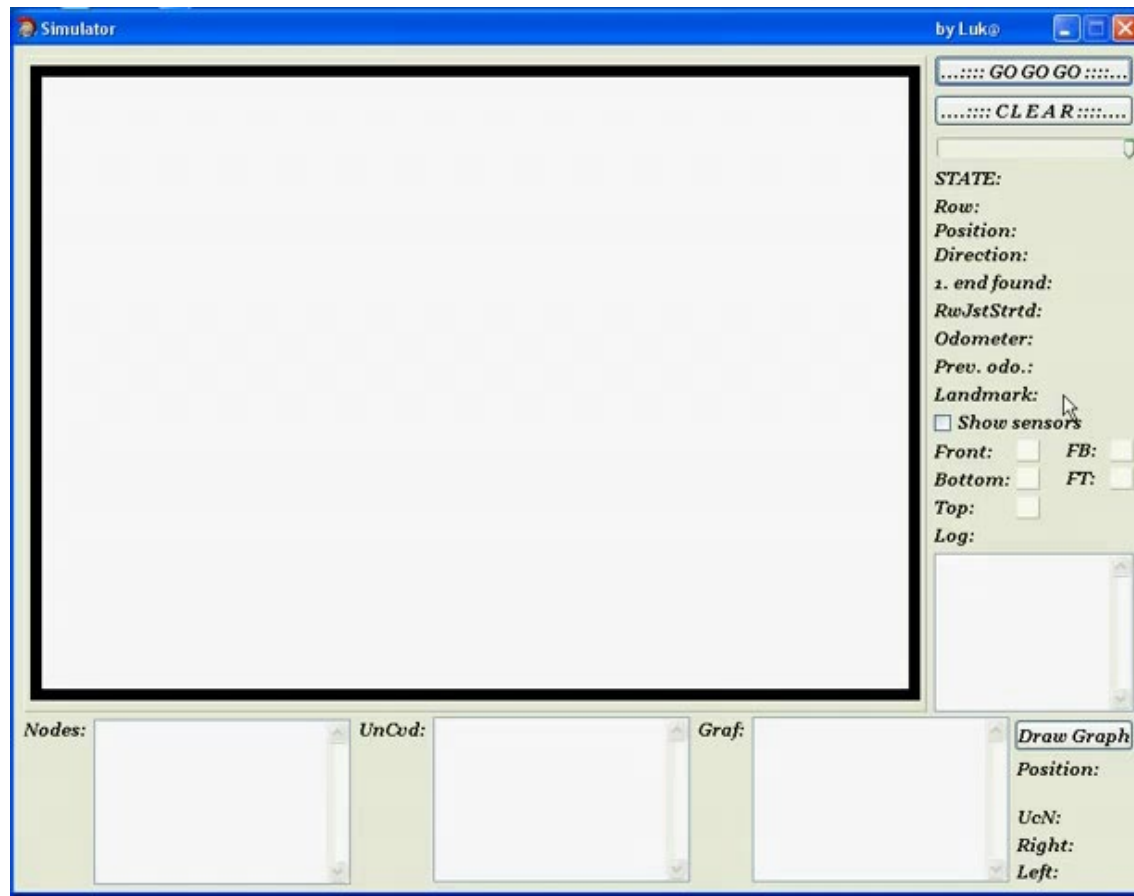
Algoritmus nájdenia kritických bodov

1. **Dopredná fáza** – robot sleduje rez, kým nenarazí na prekážku, potom sleduje hranicu prekážky, kým sa od rezu nevzdiali o šírku robota alebo nenarazí na kritický bod
2. **Reverzná fáza** – jedno alebo viac kôl v opačnom smere, koniec ak narazí na kritický bod alebo celkový pohyb prekročí šírku robota
3. **Uzatváracia fáza** – jedno alebo viac kôl v smere rezu spolu so sledovaním hraníc prekážok, koniec ak narazí na štartovací bod

Algoritmus nájdania kritických bodov



EBD v neznámom prostredí

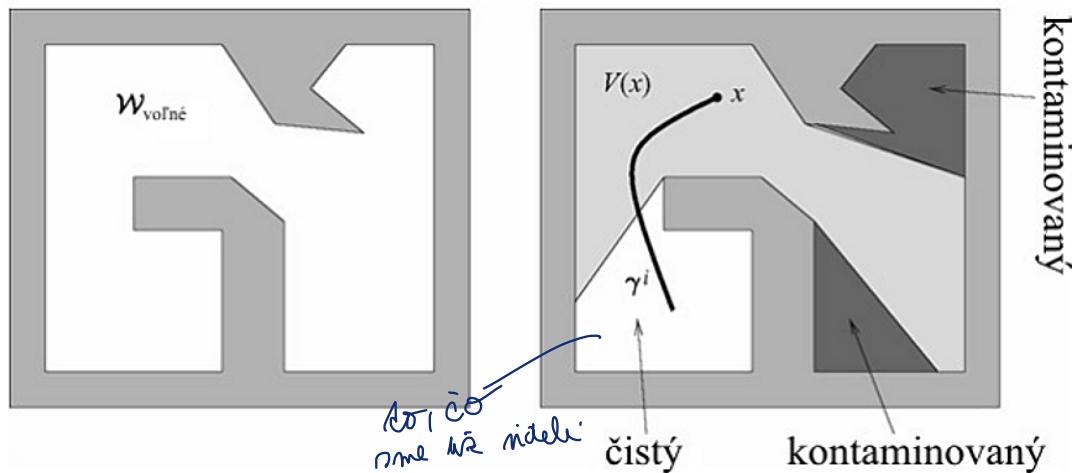


EBD metódou viditeľnej dekompozície

↳ diaľková funkčná dekompozícia

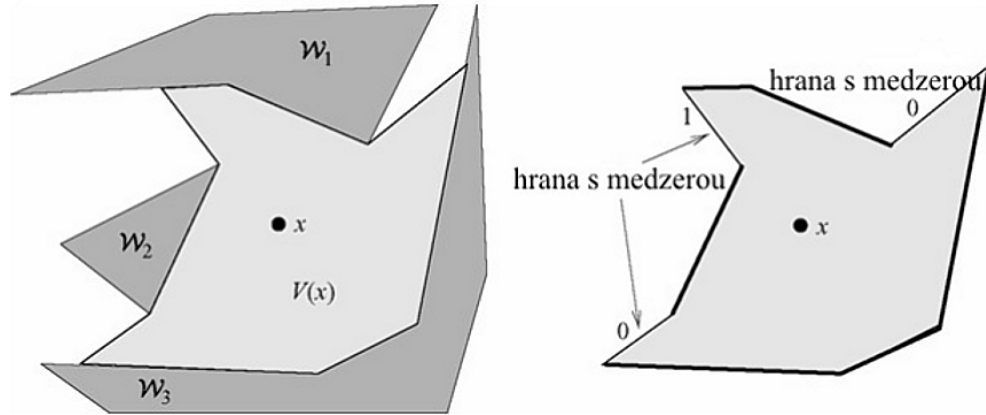
|| motivácia - poliacajna!
vzda - chytanie
slodzia ||

- viditeľnosť časti prostredia
- prostredie je rozdelené na regióny kontaminované (ešte nepreskúmané), zjavné (už preskúmané) a znovu kontaminované (zmena v prostredí)



Typy hrán

1. charakterizujúce obrysy prekážok
2. charakterizujúce náhlu zmenu v prekážke, resp. skok v meranej vzdialenosti k prekážke = medzera medzi dvoma prekážkami, tzv. **hrana s medzerou**



EBD metódou viditeľnej dekompozície

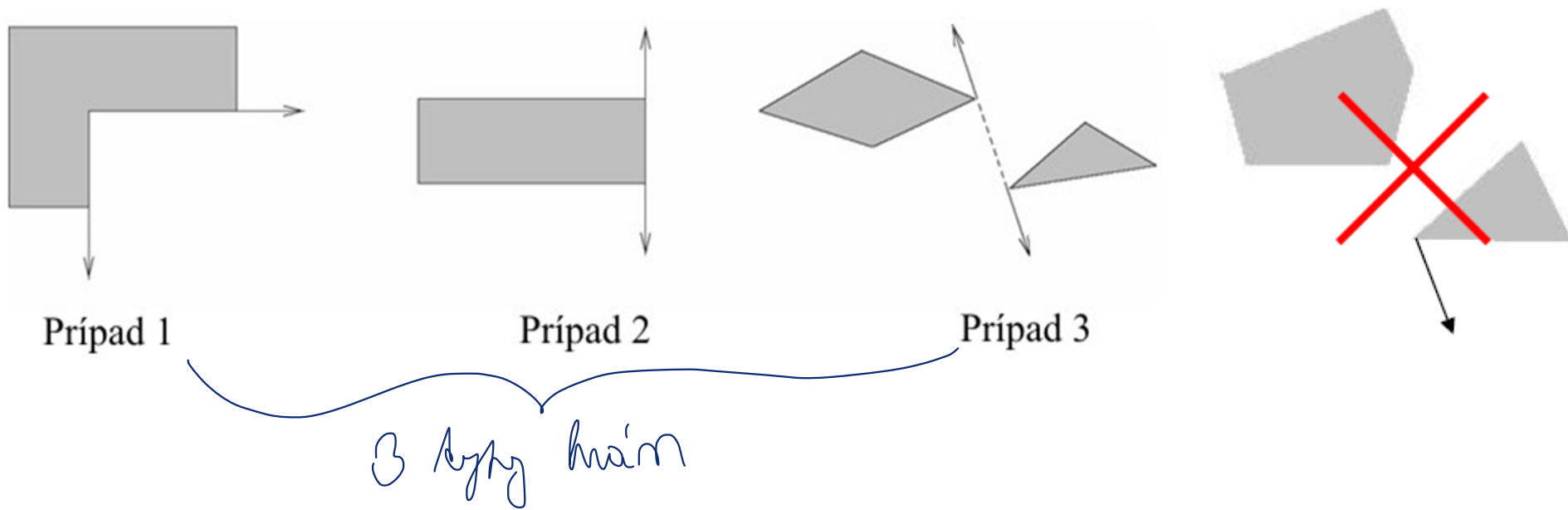
- ak sú dva vrcholy navzájom viditeľné (t. j. ich spojnicu nepretína žiadna prekážka), treba z nich vytvoriť dve čiary → čiara je orientovaná z jedného vrcholu v smere spojnic vrcholov smerom od opačného vrcholu, t. j. pre vrchol v_i z prekážky O_i a pre vrchol v_j z prekážky O_j , ak platí:

$$\lambda v_i + (1 - \lambda) v_j \in Q_{volne} \quad \forall \lambda \in (-\varepsilon; 1 + \varepsilon)$$

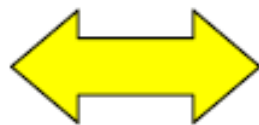
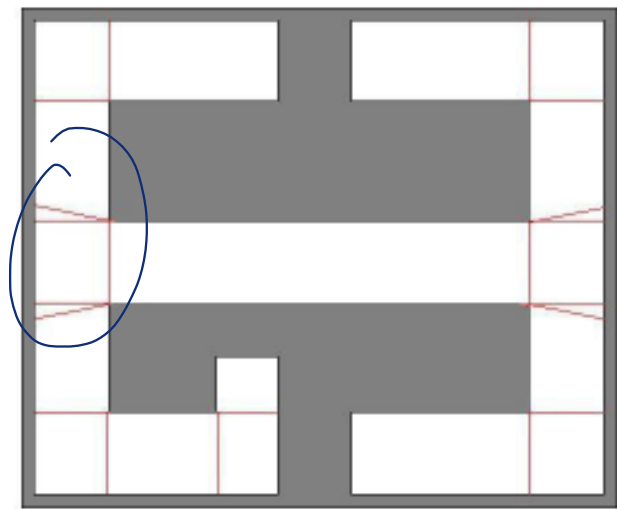
pre nejaké $\varepsilon > 0$, potom predĺž čiaru z v_i smerom od v_j a naopak, kým čiara nepretne prekážku

EBD metódou viditeľnej dekompozície

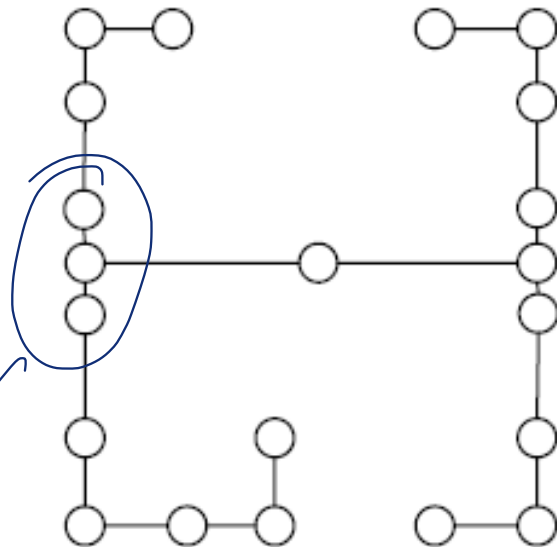
- potom predĺž čiaru z v_i smerom od v_j a naopak, kým čiara nepretne prekážku:



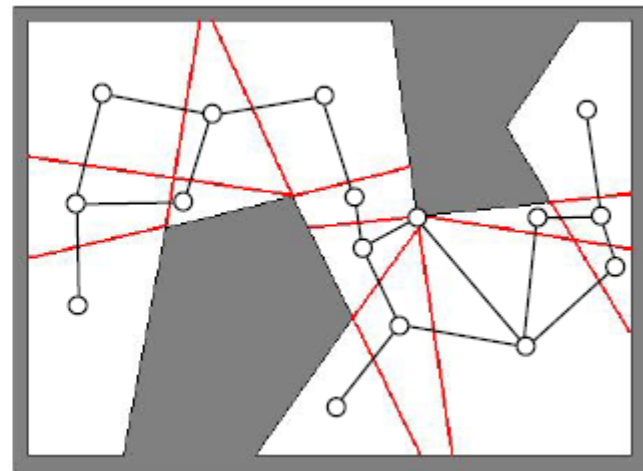
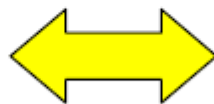
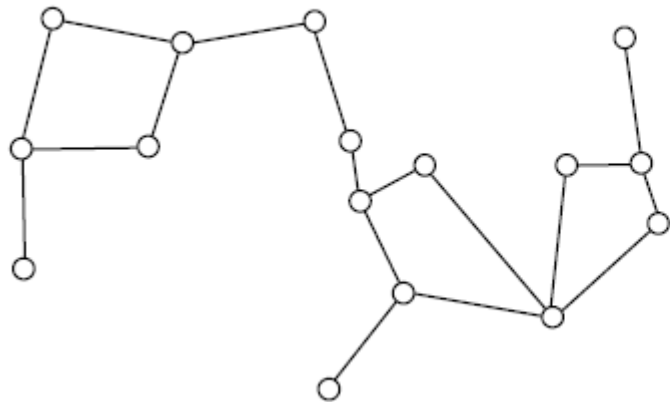
EBD metódou viditeľnej dekompozície



*krížovky
(hudebná
konológia)*



EBD metódou viditeľnej dekompozície



Exaktná bunková dekompozícia (EBD)

- veľká výpočtová náročnosť, ak prostredie obsahuje veľa objektov
 - vhodná pri prostredí s veľkými rozmermi a malým počtom objektov
- 

Približná bunková dekompozícia

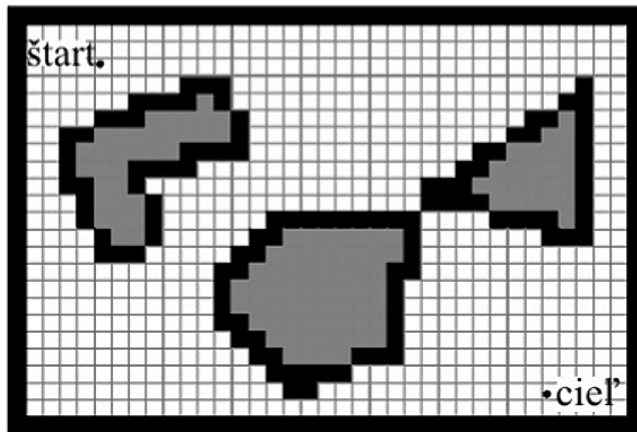
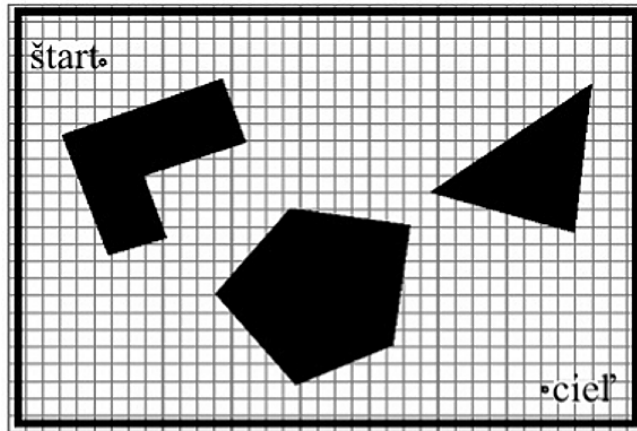
(PBD) *lepšia pri veľkom počte objektov*

- údaje zo snímačov sa v robotike najľahšie reprezentujú pomocou mriežky → najčastejšie používaná
- mriežka:
 - pevná veľkosť bunky
 - premenlivá veľkosť bunky

PBD s pevnou veľkosťou bunky

- veľkosť bunky je nezávislá od objektov v prostredí, volí sa → niektoré vlastnosti prostredia sa neobjavia, tzv. **chyba diskretizácie**
- nenáročné plánovanie dráhy mobilného robota
- výpočtová náročnosť nezávisí od počtu objektov ani od ich komplexnosti
- potreba veľkého úložného miesta (aj napriek malému počtu objektov v prostredí)

PBD s pevnou veľkosťou bunky



PBD s premenlivou veľkosťou bunky

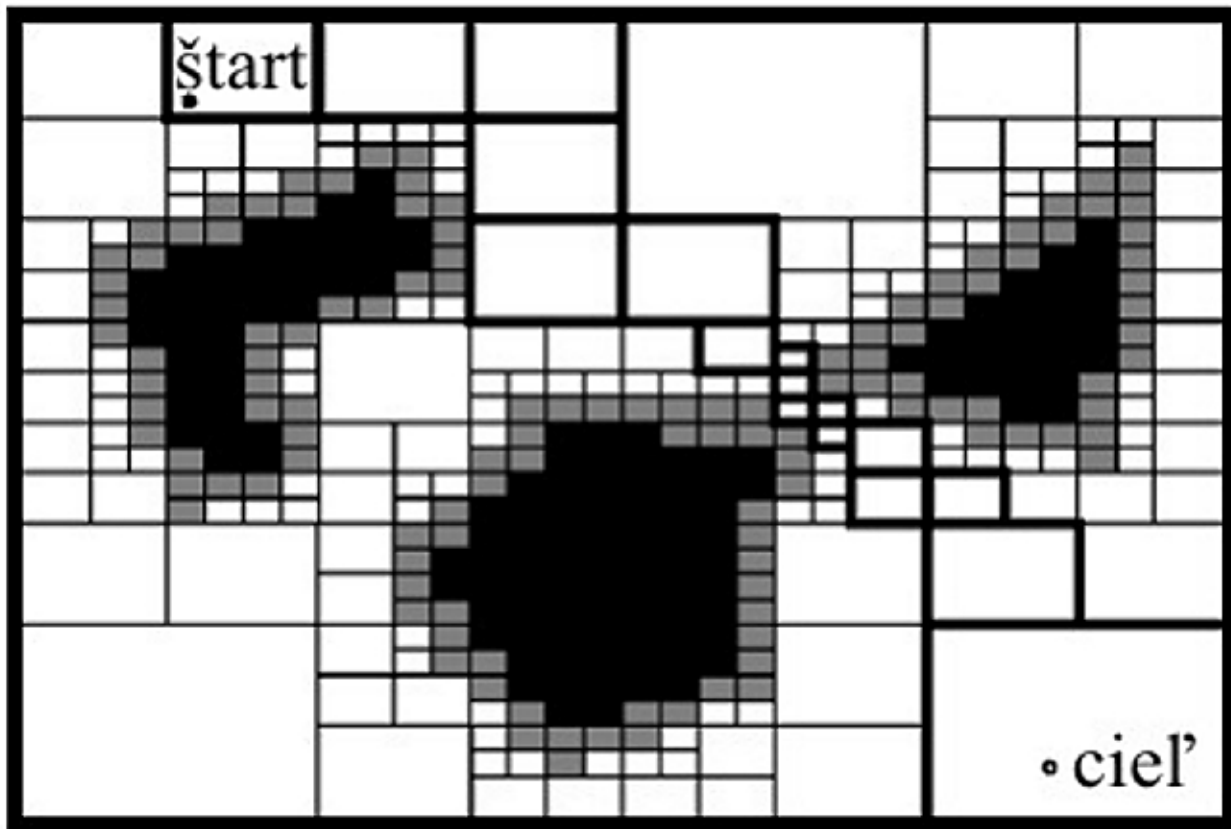
- **definícia:** rekurzívna mriežka, kde každá bunka má vopred určenú minimálnu veľkosť odvodenú od veľkosti robota alebo jej veľkosť je odvodená od štyroch buniek inej veľkosti
- napr. kvadrantový strom

Kvadrantový strom

1. vložená bunka veľkosti celej mapy
2. test, či nejaký objekt zasahuje do tejto bunky;
3. ak áno, táto bunka je rozdelená na štyri rovnomerné časti, ktoré sú potom rovnakým spôsobom otestované
4. pokračuje dovtedy, kým nie sú všetky bunky rozdelené na voľné a obsadené, alebo keď veľkosť najmenších odvodených buniek je menšia alebo rovná veľkosti robota

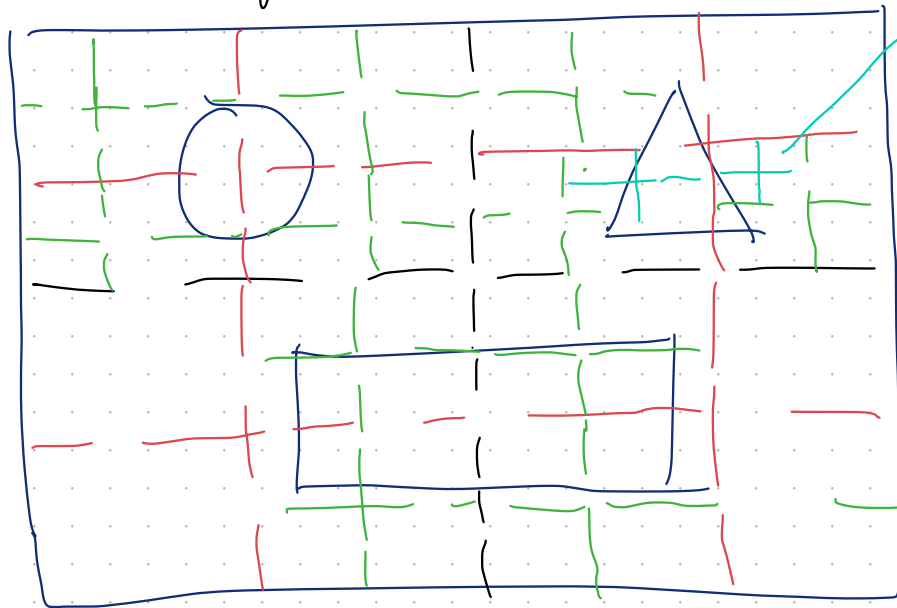
SKÚŠKA?

Kvadrantový strom



Na stránke: Radenie $\rightarrow$ svoje si 3 pechačky a značky
strom do 4. stromu

$\Rightarrow$ rady delíme lanky na 4 časti!



4. úroveň
akože sme
delíme
obsadené
lanky

akože sme
nemusíme
deliť, ale
a týmto spôsobom
sme nie je lanka
obsadená!

Kvadrantový strom



Z metrických na topologické

⇒ kóli zjednodušeniu miestom → kólie plánovanie
Projektovia

Tvorba máp z metrických na topologické

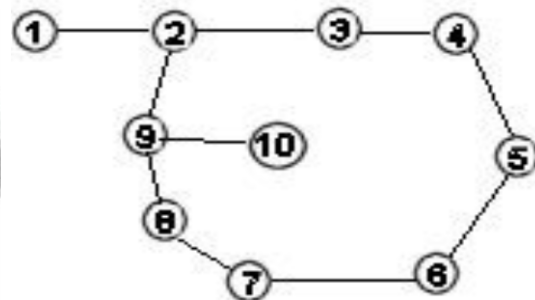
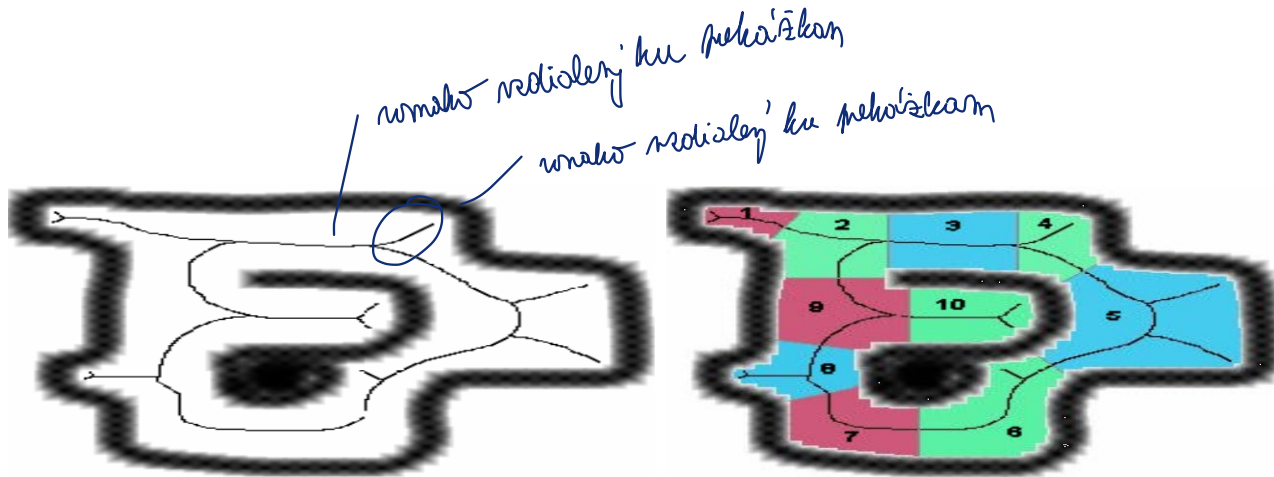
- metrické mapy sa ľahko konštruujú a sú dobre použiteľné na vlastnú lokalizáciu v prostredí
- pomocou topologických máp je možné rýchlo naplánovať cestu a dráhu
- Thrunova metóda
- kognitívna hierarchická mapa

ako nahradiť na
vnútri shi'osť
ale metódy

Thrunova metóda

1. Určiť, ktoré bunky sú obsadené, a ktoré nie pomocou prahovania pravdepodobnostných hodnôt obsadenia
2. Vytvoriť Voronoiov diagram (sústava neobsadených bodov, ktoré majú aspoň dva základové body v rovnakej vzdialenosti, t. j. najbližšia obsadená bunka)
3. Identifikovať kritické body a čiary; kritický bod je súčasťou Voronoiovho diagramu ak existuje vzdialenosť $\varepsilon > 0$, pre ktorú vzdialenosť zo všetkých bodov k základovým bodom v ε okolí bodu $[x, y]$ nie je menšia ako ε
4. Vygenerovať kritické čiary spojením kritických bodov s ich základovými bodmi = rozdelenie na vrcholy topologickej mapy

Thrunova metóda



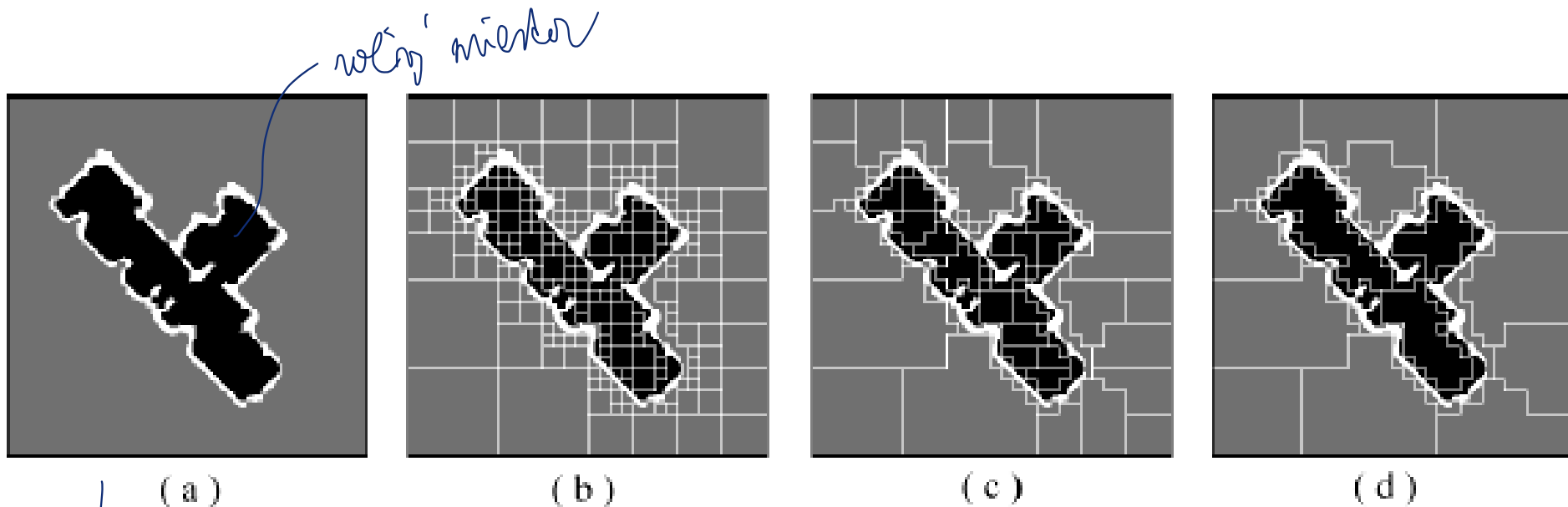
Kognitívna hierarchická mapa

0 - voľná 1 - obsadená
0,5 - nepreskúmaná

1. prahovanie metrickej mapy

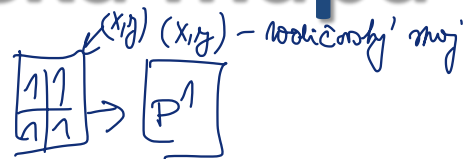
- bunka, ktorej hodnota obsadenia je menšia ako prah U_1 je voľná
- bunka, ktorej hodnota obsadenia je viac ako U_1 , ale menej ako U_2 , je nepreskúmaná
- bunka, ktorej hodnota obsadenia je viac ako U_2 je obsadená
- vzniká tzv. báza pyramidálnej štruktúry

Kognitívna hierarchická mapa



↳ 1. nahrazenie metrickej mapy

Kognitívna hierarchická mapa



2. generovanie hierarchickej štruktúry

- každá úroveň / pyramídy reprezentuje $\frac{1}{4}$ buniek priamej nižšej úrovne
- pre každú bunku pyramídy (x,y,l) je asociovaných 5 parametrov:
 1. Homogenita $H(x,y,l)$ – rovná 1, ak štyri bunky priamo pod (x,y,l) na úrovni $l-1$ odpovedajú rovnakej pravdepodobnosti obsadenia a ich hodnota homogénosti je rovná 1 – *akú veľkú oblasť sme mohli*
 2. Pravdepodobnosť obsadenia $P(x,y,l)$ – ak je (x,y,l) homogénna bunka, potom $P(x,y,l)$ je rovná pravdepodobnosti obsadenia ľubovoľnej zo 4-och buniek z úrovne $l-1$; ak nie je homogénna, $P(x,y,l)$ je nastavená na fixnú hodnotu → *definujeme rozličnosť' moji'*
 3. Oblasť $A(x,y,l)$ – rovná súčtu plôch štyroch buniek z nižšej úrovne, ktoré prislúchajú bunke (x,y,l)

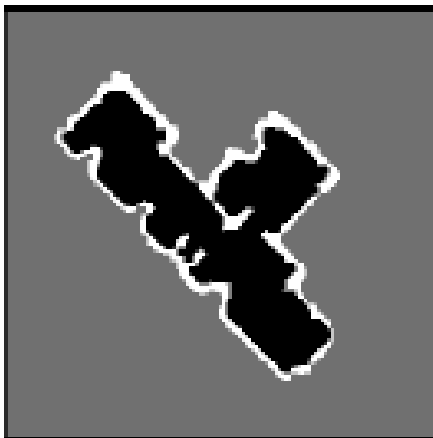


Kognitívna hierarchická mapa

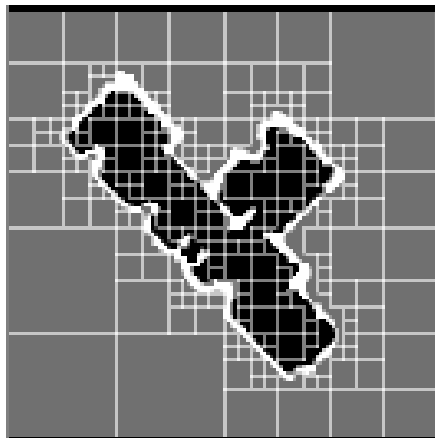
2. generovanie hierarchickej štruktúry

- pre každý uzol pyramídy (x, y, l) je asociovaných 5 parametrov:
 4. Rodičovský spoj (X, Y) (x, y, l) – ak (x, y, l) je homogénna bunka, potom rodičovský spoj štyroch buniek pod touto bunkou z úrovne $l-1$ je nastavený na (X, Y) ; inak je nastavený na 0
 5. Ťažisko $C(x, y, l)$ – ťažisko oblasti zo základnej úrovne, ktorá je spojený s uzlom (x, y, l)
- bunky na vrchných úrovniach, ktoré majú hodnotu homogénnosti rovnú 1 sú spojené do pravidelných homogénnych regiónov
- mapa je síce topologická, ale jej veľkosť závisí od rozloženia prekážok, a preto nie je optimálna

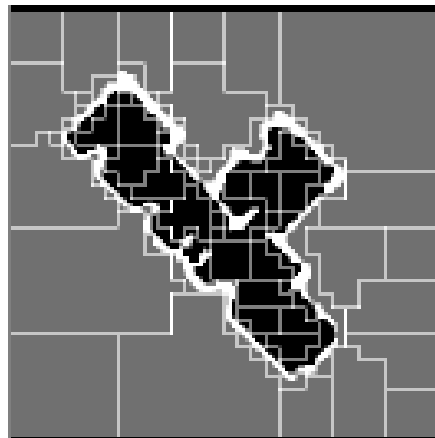
Kognitívna hierarchická mapa



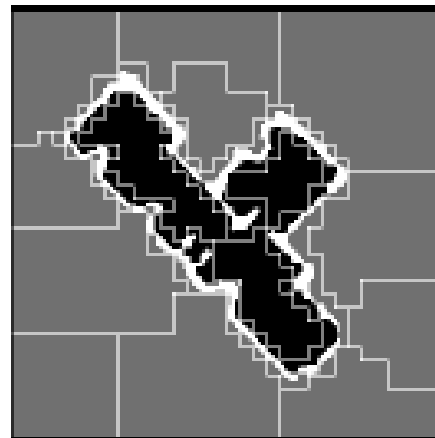
(a)



(b)



(c)



(d)

↳ generovanie hierarchickej štruktúry

Kognitívna hierarchická mapa

medanirumore! možanie

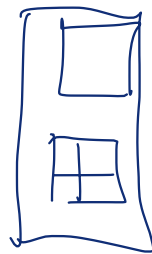
3. Fúzia homogénnych buniek bez rodičov

– bunka (x,y,l) je spojená k rodičovi susednej bunky $(x',y',l+1)$ ak spĺňa:

$$H(x,y,l)=1 \text{ a } H(x',y',l+1)=1$$

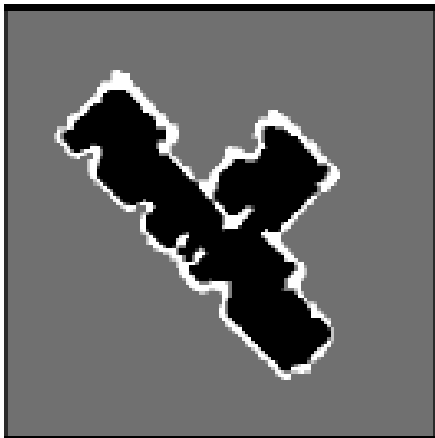
$$P(x,y,l)=P(x',y',l+1)$$

$$\| C(x,y,l) - C(x',y',l+1) \|^2 < DistMax$$

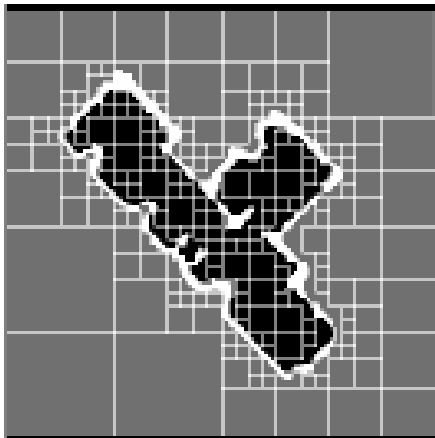


DistMax – hranica, ktorá určuje maximálny rozptyl oblastí na základnej úrovni

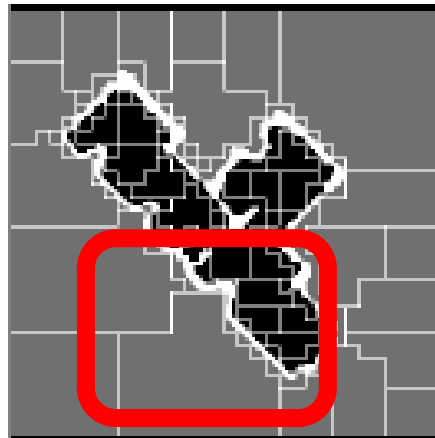
Kognitívna hierarchická mapa



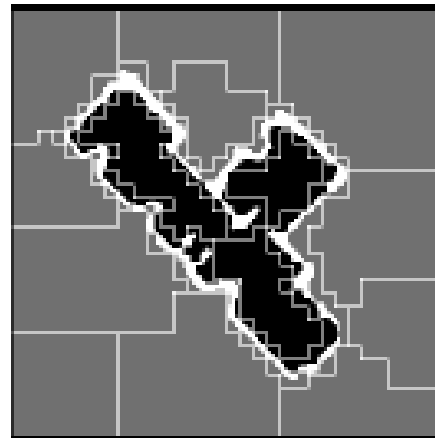
(a)



(b)



(c)



(d)

/
spojenie na rôznych úrovniach

Kognitívna hierarchická mapa

4. Fúzia susedných homogénnych buniek

— dve susedné bunky (x_1, y_1, l) a (x_2, y_2, l) sú spojené ak:

$$(X, Y) (x_1, y_1, l) = 0$$

$$(X, Y) (x_2, y_2, l) = 0$$

$$H(x_1, y_1, l) = 1 \text{ a } H(x_2, y_2, l) = 1$$

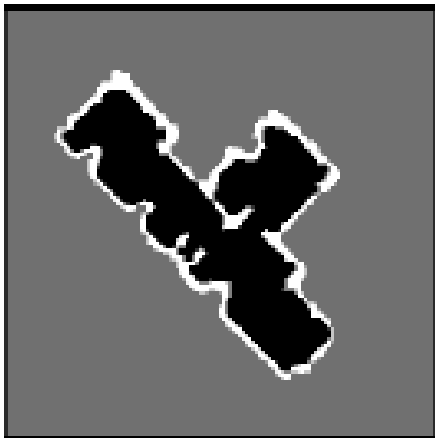
$$P(x_1, y_1, l) = P(x_2, y_2, l)$$

$$\|C(x_1, y_1, l) - C(x_2, y_2, l)\|^2 < \text{DistMax}$$

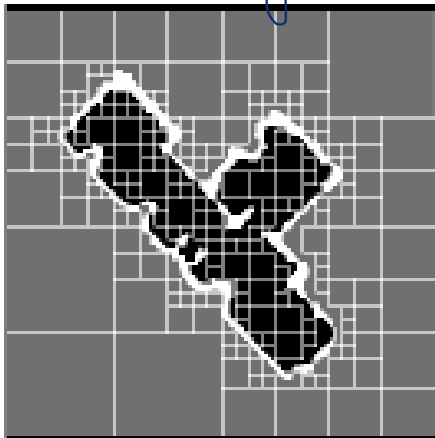
májanie
buniek,
ktorej sú rovnaké
u'mne

Kognitívna hierarchická mapa

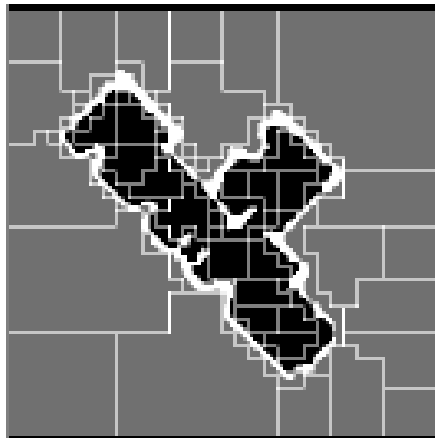
Toto nebude na skúške chcieť
mátičky maiz



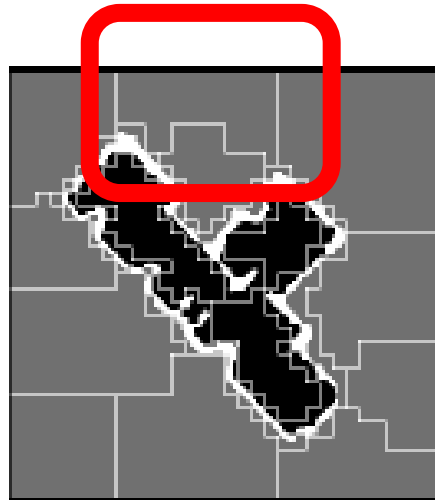
(a)



(b)



(c)



(d)

/
4. úroveň

Kognitívna hierarchická mapa

- ak sú vykonané tieto 4 kroky:
 - každá bunka s homogénnosťou 1 je spojená do homogénnych nepravidelných zón na základni
 - bunka, ktorá má homogénnosť rovnú 1 sa stane vrcholom topologickej mapy, ak nie je spojená k rodičovskej bunke, ktorá má homogénnosť rovnú 1
- mapa je reprezentovaná minimálnym počtom vrcholov
- možno robiť on-line, pretože prepočet topologickej mapy nastáva vždy, keď sa význačne zmení metrická mapa

Globálna navigácia – plánovanie dráhy

Globálna navigácia (GN)

- **definícia:** súbor úloh, ktoré pomáhajú nájsť cestu/dráhu medzi štartom a cieľom na základe stanoveného kritéria alebo podmienok (dĺžka dráhy, čas pohybu, spotreba energie a pod.)
- obvykle minimalizácia alebo maximalizácia kritéria alebo podmienok
- dráha robota stanovená pomocou globálnej navigácie pozostáva z množiny bodov, medzi ktorými existujú bezkolízne priechody

Globálna navigácia

- nazývaná aj ako plánovanie (z angl. path planning) → nutnosť použiť komplexnejšie vyhodnotenie údajov
- odpovede na otázky:
 - Ako sa tam dostanem?
 - Kde som už bol?
 - Aká je najlepšia cesta do cieľa?
- reálny robot potrebuje súčinnosť globálnej navigácie s reaktívnou navigáciou a s lokalizáciou!

Globálna a reaktívna navigácia

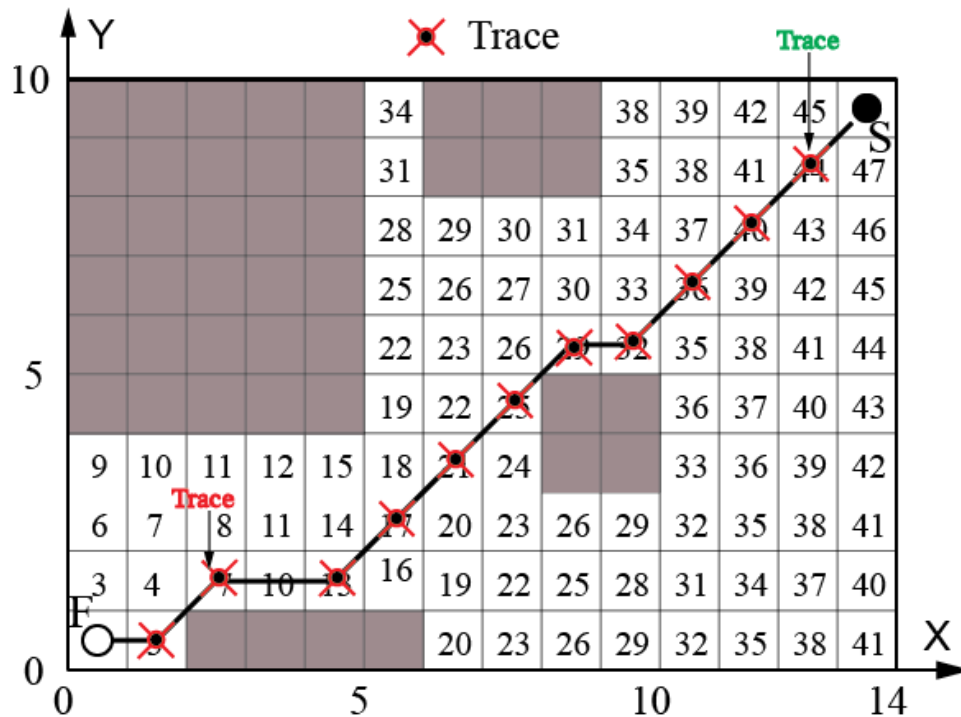
- ak má robot len globálnu navigáciu → nezvládne samostatne dosiahnuť vzdialený cieľ
- ak má robot len reaktívnu navigáciu → nezvládne fyzicky dosiahnuť cieľ

Cesta - Hovorí, že medzi A a B existuje spojenie

Dĺžka - Hovorí, že aká je dĺžka cesty = vzdialenosť

Trajektória - má časový rozměr = "za koľko to pojedeme" !

Globálna a reaktívna navigácia



Globálna navigácia a zmeny v prostredí

- off-line
 - predpokladá sa nemenné prostredie
 - pri zmene treba znova napláňovať dráhu
 - väčšina metód
- on-line
 - pri zmene dôjde len k preplánovaniu medzi aktuálnou pozíciou a cieľom, prípadne len čiastočná úprava dráhy

GN s topologickou mapou

GN s topologickou mapou

- kvalitatívna navigácia podobná tej ľudskej (napr. chod' touto halou, zaboč doľava, chod' do druhej miestnosti a pod.)
- spočíva v nájdení spojenia vrcholov štartu a cieľa v topologickej mape na základe zvoleného optimalizačného kritéria
- často sa používajú topologické mapy s ohodnotenými hranami = istá metrika medzi jednotlivým vrcholmi (čas prechodu, dĺžka prechodu a pod.)

GN s topologickou mapou

- nízka náročnosť na výpočtový výkon a pamäť
- jednoduché plánovanie cesty a dráhy robota prostredím
- nenáročné rozširovanie o nové miesta
- neschopnosť reprezentovať metrické vlastnosti prostredia
- nutnosť neustále porovnávať a korigovať reálne väzby v mape

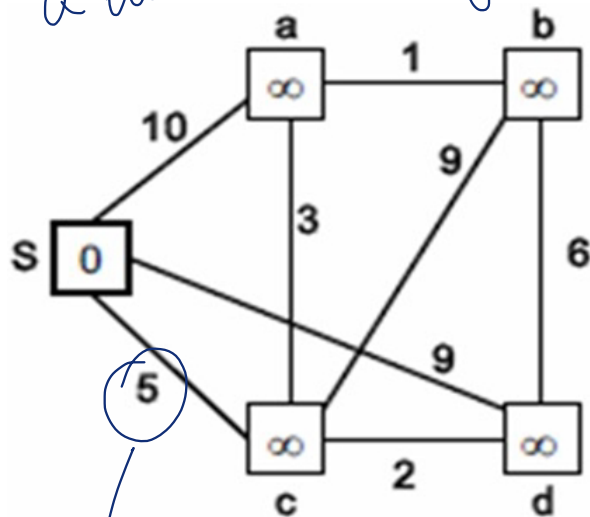
Dijkstrov algoritmus

1. Inicializuj množinu *Pripravené* – vlož do nej štartovný vrchol. Množina *Pripravené* charakterizujú vrcholy, pre ktoré už bol, resp. je výpočet algoritmu vykonaný.
2. Vyber vrchol s najmenšou vzdialenosťou k množine *Pripravené*. Pre susedné vrcholy k vybranému vrcholu vypočítaj vzdialenosti od štartového vrcholu cez vybraný vrchol. Ak je táto vzdialenosť menšia ako už uložená vzdialenosť, nahraď ju novou a nahraď predchodcu pre testovaný vrchol vybraným vrcholom.
3. Pridaj vybraný vrchol do množiny *Pripravené*.
4. Opakuj kroky 2 a 3, až kým nie sú v množine *Pripravené* všetky vrcholy grafu.

Dijkstrov algoritmus – príklad

budeme hrúbiť
a hodnotiť mapy

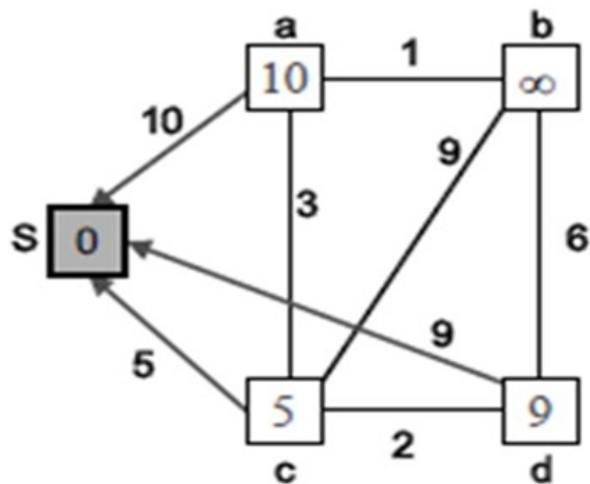
na začiatku môže byť



Z S do:	S	a	b	c	d
Vzdialenosť	0	∞	∞	∞	∞
Predchodca	-	-	-	-	-
Pripravené	{ }				

Keďže
čísla "vzdialenosti"
si budeme na začiatku voliť rovné

Dijkstrov algoritmus – príklad

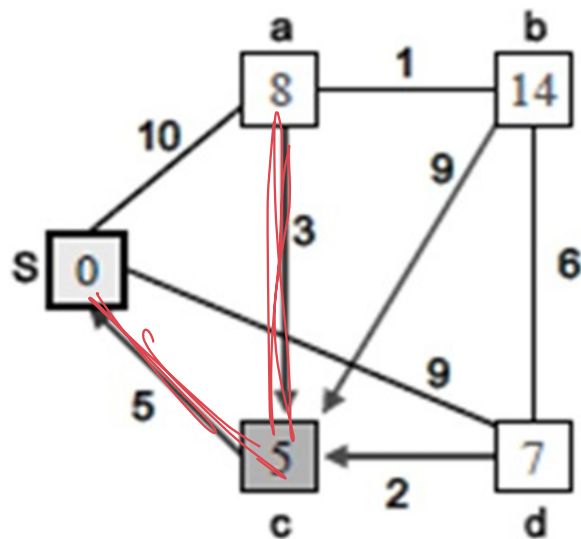


neskuseli o S

možná zmeniť hodnotu

Z S do:	S	a	b	c	d
Vzdialenosť	0	10	∞	5	9
Predchodca	-	S	-	S	S
Pripravené	{S}				

Dijkstrov algoritmus – príklad

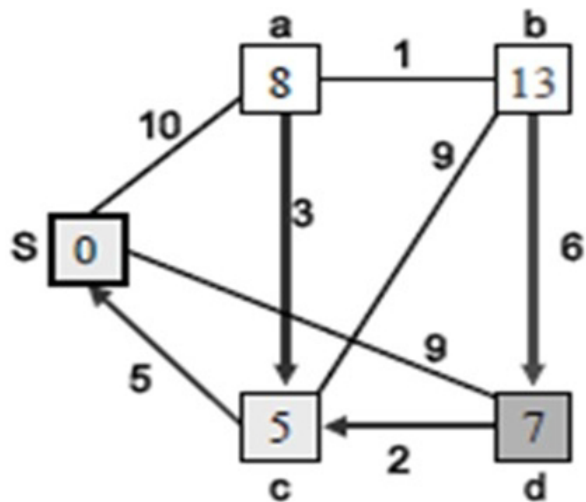


Z S do:	S	a	b	c	d
Vzdialenosť	0	8	14	5	7
Predchodca	-	c	c	S	C
Pripravené	{S, c}				

najmenší
ktorý
môže

lebo c je to
najmenšie z S

Dijkstrov algoritmus – príklad

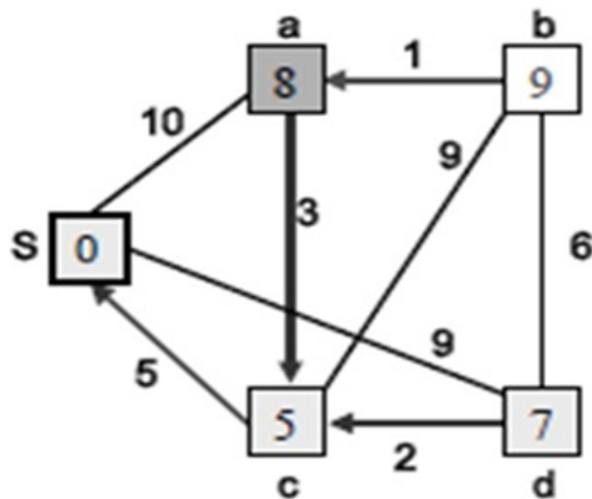


Z S do:	S	a	b	c	d
Vzdialenosť	0	8	13	5	7
Predchodca	-	c	d	S	C
Pripravené	{S, c, d} - daime				

normálnosť je
sam seba
ok

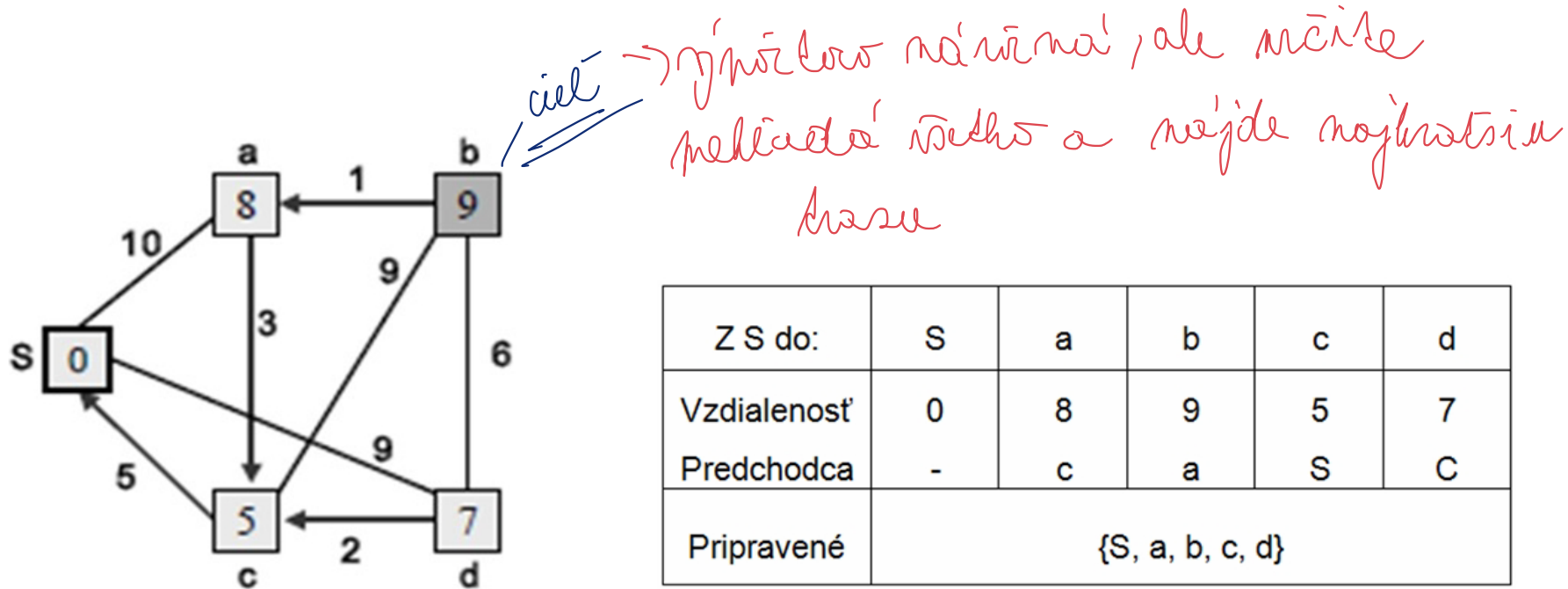
ho do
miponere

Dijkstrov algoritmus – príklad



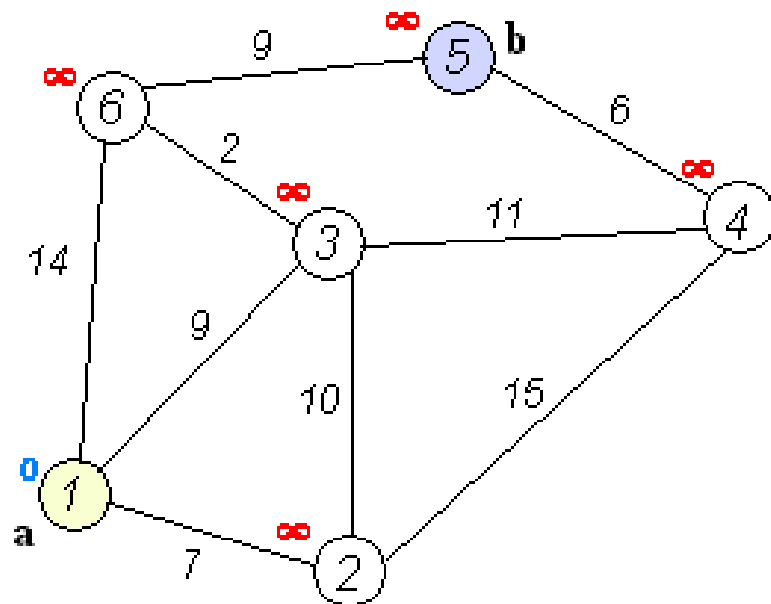
Z S do:	S	a	b	c	d
Vzdialenosť	0	8	9	5	7
Predchodca	-	c	a	S	C
Pripravené	{S, a, c, d}				

Dijkstrov algoritmus – príklad

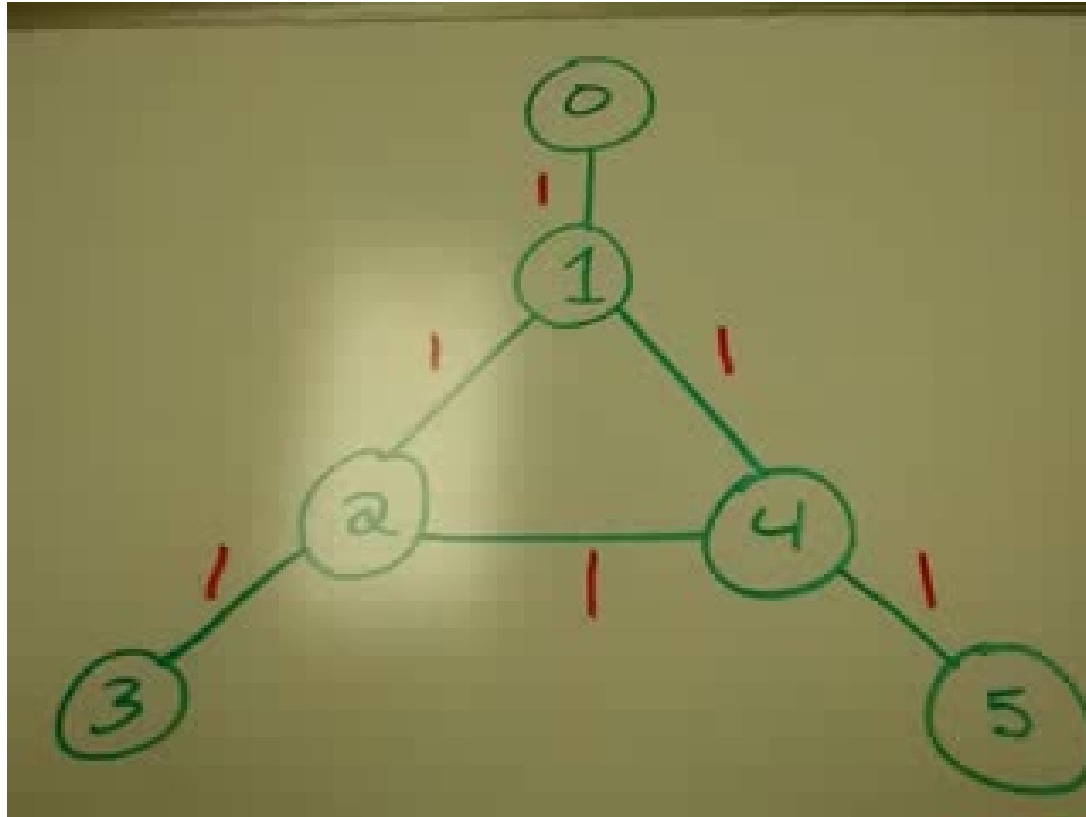


$$pre[b] = a; pre[a] = c, pre[c] = S \quad S \rightarrow c \rightarrow a \rightarrow b$$

Dijkstrov algoritmus – príklad



Dijkstrov algoritmus s iRobot Create



A* algoritmus

— každý vrchol
má heuristiku

- expanduje vrcholy priechodné od štartu, resp. od aktuálne expandovaného vrcholu
- minimálna vzdialenosť do expandovaného vrcholu je určená ako súčet ohodnotených hrán celkovej dráhy do vrcholu a heuristickej (Euklidovskej, Čebyševovej, Manhattskej) vzdialenosti vrcholu od cieľového vrcholu (t. j. odhad zostávajúcej vzdialenosti do cieľa)
- z potenciálnych kandidátskych vrcholov sa vyberá ten s najmenším ohodnotením = algoritmus „best-first“



A* algoritmus

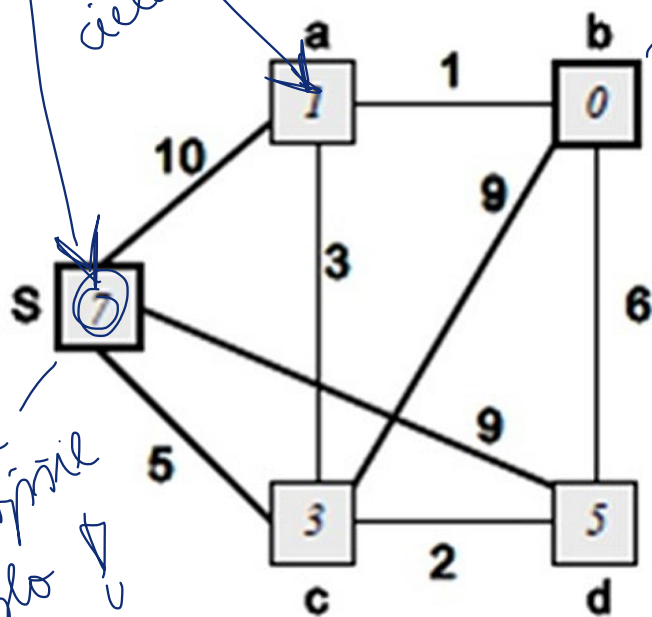
- rozdiel oproti Dijkstrovému algoritmu:
 - vyžaduje aj znalosť heuristickej vzdialenosti vrcholov od cieľového vrcholu (napr. vzdialenosť vzdušnou čiarou alebo lineárna vzdialenosť)
 - z aktuálneho vrcholu prehľadáva všetky voľné cesty
 - môže sa zdať výpočtovo náročnejší (potreba ukladať nekompletné dráhy a ich dĺžky), to je však možné riešiť použitím rekurzívnych algoritmov
 - kvalita odhadu heuristickej vzdialenosti vrcholov od cieľa zvyšuje rýchlosť výpočtu algoritmu

A* algoritmus

PLŮTKA

Ka' ika' konvojichal mapu
ale i ne' shodnudenii

= ciet je na 0



start
je nejmenší
číslo !

číslo volíme samy !

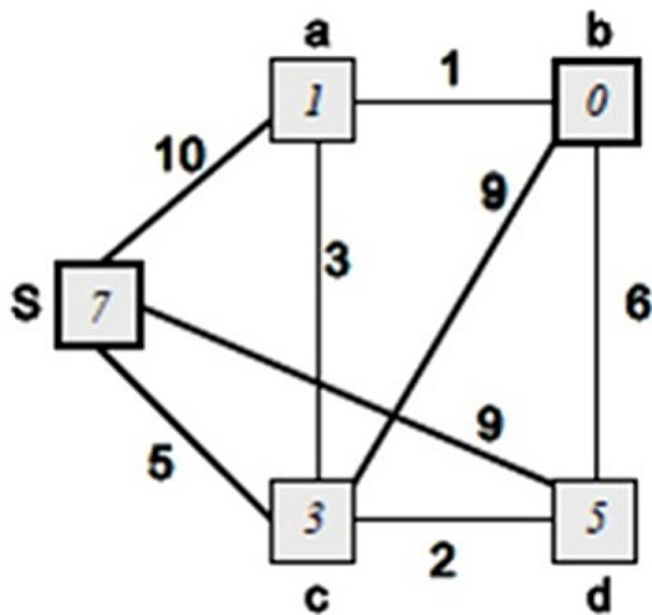
A* algoritmus

záleží na
rozdělení
váhových a do
cíle

$$\{S, a\} = 10 + 1 = 11$$

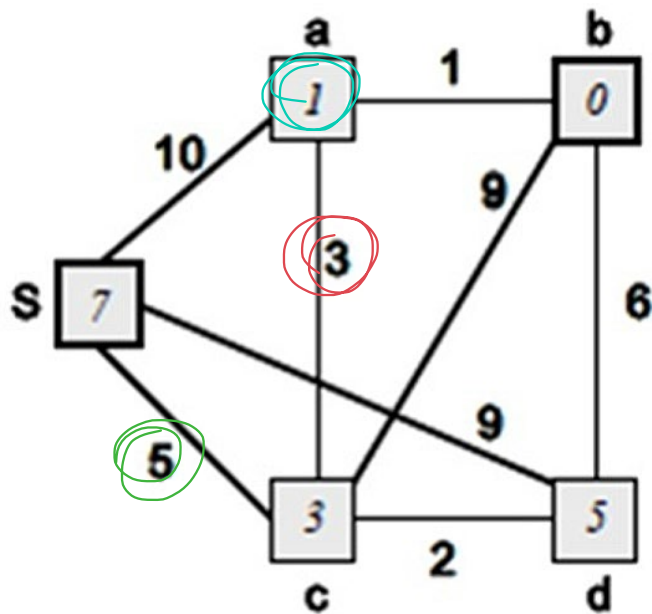
$$\{S, c\} = 5 + 3 = 8$$

$$\{S, d\} = 9 + 5 = 14$$



A* algoritmus

- posrám rdy
najmensie glovine



$$\{S, a\} = 10 + 1 = 11$$

$$\{S, c\} = 5 + 3 = 8$$

$$\{S, d\} = 9 + 5 = 14$$

$$\{S, c, a\} = 5 + 3 + 1 = 9$$

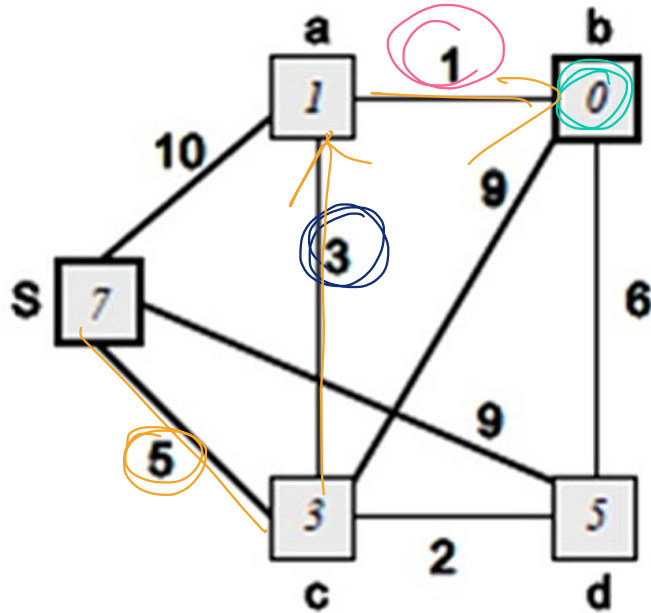
$$\{S, c, b\} = 5 + 9 + 0 = 14$$

$$\{S, c, d\} = 5 + 2 + 5 = 12$$

redušná
redušnosť
hde som
chvilu
od cieľa

pre redušnú
redušnosť

A* algoritmus



$$\{S, a\} = 10 + 1 = 11$$

$$\{S, c\} = 5 + 3 = 8$$

$$\{S, d\} = 9 + 5 = 14$$

$$\{S, c, a\} = 5 + 3 + 1 = 9$$

$$\{S, c, b\} = 5 + 9 + 0 = 14$$

$$\{S, c, d\} = 5 + 2 + 5 = 12$$

$$\{S, c, a, b\} = 5 + 3 + 1 + 0 = 9$$

- najmenšie
a druhom
kolom
susedne
a to je b

1. kroki $(S, a) = 11$
 1. VB $(S, c) = 8$
 $(S, d) = 14$

2. kroki $(S, a, x) = 22$
 $(S, a, y) = 25$
 $(S, a, z) = 26$

mekar je do 2. VB

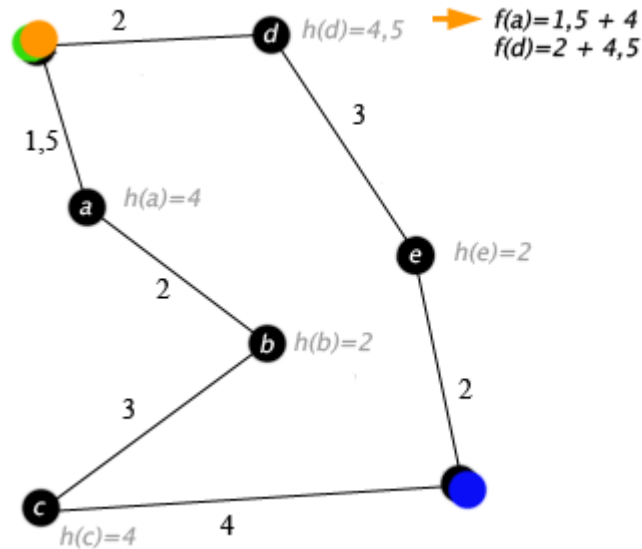
⇒ relke čísla,
 + náhne
 sa na

ale 2 kroky
 nemám
 menšie čísla
 ako odp. som 11

2. kroki $(S, c, d) = 13$
 3. VB $(S, c, b) = 14$
 najmenšie
 číslo
 potom $(S, c, a) = 16$

VB = y'ker

A* algoritmus – príklad



A* algoritmus v iných aplikáciách



Porovnanie Dijkstra a A*

- <https://clementmihailescu.github.io/Pathfinding-Visualizer/#>
- Kde všade je MATLAB?

Koniec

Ďakujem za pozornosť